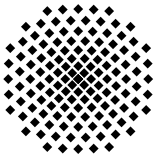


University of Stuttgart
Institute for Signal Processing and System Theory
Professor Dr.-Ing. B. Yang

Bachelorarbeit S1509

Implementation of an Electromyography Processing Pipeline from Torch model to TinyML on resource-constrained microcontroller platforms

Arbeitstitel, to be defined (TBD)

Author: Andrew Fady Fahmy

Date of work begin: 01/03/2026
Date of submission: 30/06/2026

Supervisor: Jonas Koellner, Prof. Dr Turker Ince

Keywords: sEMG, TinyML, Digital Signal Processing, Deep Learning, regression, Limited-Resource Microcontroller Platforms

Abstract TBD

Contents

1. Basics and Foundations

1.1. EMG and Deep Learning Pipelines for Bioelectric Signal Processing

1.1.1. Bioelectric Signals and Human Machine Interaction (HMI)

Bioelectric signals are simply defined as a biological signal that originates from a human body, they are the sum of action potentials of cells at an anatomical site such as the heart, brain, or skeletal muscle [1]. These signals have been used extensively in Human Machine Interaction as they provide key insights about human's physiological activity as well as presenting a more natural interaction medium between humans and computers. Although our primary focus in this work is about Electromyography (EMG) signals, we will frequently discuss other commonly researched bioelectric signals such as EEG (Electroencephalogram) and ECG (Electrocardiogram) signals given the abundance of useful Processing pipelines and HMI research conducted using these 2 domains.

1.1.2. Electromyography (EMG) Basics

Electromyography is the recording of electrical activity in muscle. These electrical impulses originate from the spinal cord and propagate all the way to the peripheral nerves activating muscular contraction [2]. The motor neurons and the muscle fibers they innervate are defined as Motor Units which are recruited by these electrical impulses producing electrical signals. Professional electromyographers analyze characteristics features of these signals (i.e waveforms, firing rates, frequency of abrupt discharges, etc) deriving useful diagnostic information to identify abnormalities and different neural and muscle diseases

The recording of EMG signals is mainly classified into two main categories. The first is intramuscular electromyography (iEMG) which is an invasive technique that includes inserting concentric needle electrodes inserted directly into muscle. The other is surface electromyography (sEMG) which is a non-invasive technique involving the placement of adhesive surface electrodes on skin surface. While iEMG has proven to be more effective in diagnosing medical conditions and abnormalities, sEMG dominate the HMI and gesture recognition sector due to its non-invasive nature, wider accessibility without the need of trained experts and user comfort [3].

In this study, we exclusively focus on sEMG data for the reasons mentioned above.

1.1.3. EMG Signal Preprocessing and Feature Extraction

Raw EMG signals suffer from a multitude of issues that obscure its insightful value. EMG signals are susceptible to various types of noise, which include electrical interference, physiological signals from adjacent muscles that can contaminate the signal, the inherent instability of the signal, which arises because the firing rate of the motor units is itself quasi-random [4] and obscure relevant information as well as movement artifacts and variations from one subject to another [3, 5].

These problems clearly motivate extensive pre-processing steps to be taken in order to extract as much insightful information as possible from EMG signals [6].Typically, sEMG signals recorded by commonly available sensors are in the range of 0 to 400 Hz. Band-pass butter worth filters of 4th order, with 20–450 Hz range, are used by most studies to retain viable information and discard the 5-20 Hz corner frequencies[6].

To filter out undesired high frequency noise. Many studies recommend the use of further smoothing algorithms like forward–backward filtering and the use of rectifiers to convert bipolar signal into positive-only mono-polar data. Normalization is also ubiquitous for digital signal processing and machine learning in order to ensure steady and fast training times as well as reducing amplitude variability across subjects, sessions, and electrode placements [5].

Windowing algorithms are widely adopted in EMG Preprocessing pipelines with varying parameter choices across the scientific literature. For real-time closed-loop control, input latency is critical: a maximum latency of 300 ms was initially recommended [7], although more recent studies suggest that it should optimally be kept between 100 and 250 ms [5, 8]. While larger window sizes are preferred for higher quality models with better accuracy, smaller window sizes are the main choice for real-time systems and edge solutions that prioritize faster performances and speed with limited resources.

Feature extraction constitutes a further preprocessing stage emphasized across both research papers and textbooks. Extracted features are categorized in DSP research into two main categories. Time Domain Features are features extracted from the filtered time-series data directly. Features like the mean absolute value (MAV), waveform length (WL), number of zero crossings (ZC), root mean square (RMS) are very widely adopted in research with varying successful contributions from each. Frequency-domain features, by contrast, are derived from the Fourier transform of the time series and include measures such as the mean frequency (MNF) and peak frequency (PKF) [9].

In this work, signal preprocessing is restricted to the time domain. The study adopts the increasingly prevalent paradigm that emphasizes feature learning through deep learning models rather than manual feature engineering, reflecting a broader shift in the field from feature engineering toward feature learning [10].

1.1.4. Machine Learning and Deep Learning Approaches for sEMG-Based Classification and Regression

Approaches to sEMG decoding fall broadly into two paradigms: classical machine learning (ML) methods, which operate on hand-crafted features extracted from the signal, and

deep learning (DL) methods, which learn task-relevant representations directly from the raw or minimally processed signal. This section surveys both, establishing the methodological context for the resource-constrained deployment that this thesis addresses.

Classical Machine Learning Approaches

Among classical ML classifiers, the Support Vector Machine (SVM) is one of the most widely adopted methods for sEMG-based gesture recognition [11]. An SVM constructs the decision boundary that maximises the margin separating the classes, where the margin is defined by the training examples lying closest to the boundary—the support vectors [11]. For data that are not linearly separable, which is typically the case for sEMG, the SVM employs the so-called kernel trick: rather than explicitly mapping the inputs into a higher-dimensional space in which a separating hyperplane exists, a kernel function (commonly the Radial Basis Function) computes the requisite inner products directly, thereby avoiding an explicit and costly nonlinear transformation [11].

SVMs have been applied extensively to sEMG classification, with feature-augmented variants reporting average accuracies as high as 99.98% on constrained gesture sets [12]. The method is regarded as one of the more robust and accurate classical classifiers for sEMG [13], and is frequently employed as the reference baseline against which newly proposed sEMG decoders—including deep models intended for embedded deployment—are evaluated [14]. Other classical classifiers recurrently reported in the literature include Linear Discriminant Analysis (LDA), k-Nearest Neighbours (kNN), and Random Forests [11], with comparative studies often finding LDA and SVM to yield broadly comparable accuracies on time-domain feature sets.

The principal limitation shared by these classical methods is their dependence on a handcrafted feature extraction and selection pipeline. Such pipelines require domain expertise to design, introduce subjectivity in the choice of features and parameters, and must frequently be re-tuned when signal characteristics change, which constrains their flexibility and generalisability [11].

Deep Learning Approaches

Deep learning circumvents the manual feature-engineering bottleneck by learning hierarchical, task-relevant representations directly from the data, enabling end-to-end classification and regression [15]. Owing to its successes across domains ranging from computer vision to digital signal processing, a broad repertoire of DL architectures has been adapted to sEMG decoding. Crucially, the choice of architecture is tightly coupled to the representation in which the sEMG signal is cast [15].

When the multi-channel sEMG is arranged spatially—treating electrode channels as a spatial axis—it can be processed as an image, making Convolutional Neural Networks (CNNs) a natural and widely used choice, mirroring their dominance in image analysis [15]. When instead the signal is treated as a set of temporal waveforms, sequence-modelling architectures become appropriate. Recurrent Neural Networks (RNNs) capture temporal dependencies by processing the sequence step by step, while their gated variants—the Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU)—introduce memory cells and gating

mechanisms that mitigate the vanishing and exploding gradient problems, thereby improving the modelling of long-range temporal dependencies [15].

Temporal Convolutional Networks (TCNs) have more recently emerged as a compelling alternative to recurrent models for sEMG sequence modelling. Because they apply convolutions along the time axis, TCNs process entire segments in parallel rather than sequentially, affording greater training efficiency and scalability than RNNs [15]. Through dilated causal convolutions, a TCN can expand its receptive field—and thus the temporal context it captures—exponentially with depth, without a commensurate increase in parameter count [14, 15]. These properties have made TCNs particularly attractive for real-time, resource-constrained sEMG decoding, a point developed further in the subsequent discussion of embedded and TinyML solutions.

1.2. TinyML and Resource-Constrained Microcontroller Platforms

1.2.1. Microcontrollers

A microcontroller unit (MCU) is a self-contained computing system integrated onto a single chip. It comprises the core components required for general-purpose computation, including a central processing unit (CPU), volatile memory (RAM), non-volatile FLASH storage, and input/output (I/O) peripherals. The adoption of MCUs has accelerated markedly in recent years, driven largely by the proliferation of IoT systems (Internet of Things), which deploy numerous interconnected devices to acquire, process, and extract actionable insights from on-ground sensor data, transmitting this information across dedicated channels and networks for subsequent processing and decision-making.

The contemporary MCU market is served by several prominent vendors, including Arduino (notably the UNO and Nano modules), Espressif Systems (the ESP32 family), and STMicroelectronics (the STM32 series), among others. This landscape continues to evolve rapidly as new architectures and capabilities emerge.

1.2.2. Limitations and Constraints of Microcontroller Platforms

The low cost, compact form factor, and minimal power consumption that characterise MCUs are achieved through significant trade-offs relative to general-purpose and high-performance computing platforms.

Foremost among these is severely constrained memory [16]. On-chip memory is approximately three orders of magnitude smaller than that of mobile devices, and five to six orders of magnitude smaller than that of cloud-based GPUs [17]. Clock frequencies are similarly limited, typically operating in the megahertz (MHz) range, in contrast to the gigahertz (GHz) range of modern general-purpose processors. A further constraint is the absence of GPUs or dedicated parallel processing units, which restricts the efficient execution of single-instruction-multiple-data (SIMD) operations and vectorised computation; this limitation has historically impeded the adoption of MCUs in the graphics and machine learning domains.

Moreover, software development for MCUs remains notoriously cumbersome, owing to fragmented vendor-specific SDKs and frameworks, the prevalent reliance on deprecated software to accommodate hardware constraints, and the limited continuity of support and maintenance for development toolchains.

Table 1.1.: Comparison of commercially available MCUs for edge ML applications.

MCU	Processor	WiFi/BT	Memory & FPU	NN / SIMD	Cost
ESP32	Xtensa LX6, 2× @ 240 MHz	Yes	520 KB SRAM; 8 MB PSRAM + 16 MB Flash (ext.); SP FPU	ESP-DL / ESP-NN	$3–8
ESP32-S3	Xtensa LX7, 2× @ 240 MHz	Yes	512 KB SRAM; 8 MB PSRAM + 16 MB Flash (ext.); SP FPU	ESP-DL / ESP-NN (vector ext.)	$4–10
Pi Pico (RP2040)	Cortex-M0+, 2× @ 133 MHz	Wi-Fi[a]	264 KB SRAM; ext. PSRAM; 2 MB Flash; no FPU	CMSIS-NN (no SIMD)	$4–6
Arduino Nano	ATmega328P (8-bit), 1× @ 16 MHz	No	2 KB SRAM; no PSRAM; 32 KB Flash; no FPU	None	$10–25
STM32H7	Cortex-M7, 1–2× @ 480 MHz	No	1 MB SRAM; ext. PSRAM; 2 MB Flash; DP FPU	CMSIS-NN/DSP (no MVE)	$8–20
STM32N6	Cortex-M55 + Neural-ART NPU, 1× @ 800 MHz	No	4.2 MB SRAM; ext. PSRAM; flashless[b]; H/SP/DP FPU	Helium SIMD + NPU (600 GOPS); ST Edge AI	$5–12

[a] Wi-Fi on Pico W only; base RP2040 has no radio. [b] No internal user flash; boots from external XSPI. NPU clocked at 1 GHz.

1.2.3. TinyML Overview

Historically, the only viable approach to machine learning (ML) and deep learning (DL) inference within IoT systems was Cloud ML. This paradigm leveraged the communication capabilities of MCUs, such as Wi-Fi, to transmit raw sensor data to remote servers possessing substantially greater computational resources for processing, subsequently retrieving the resulting commands in order to actuate physical responses. While this strategy exploited the full capabilities of cloud clusters and relaxed the hardware requirements of IoT devices, it suffers from several limitations that are particularly acute for real-time biosignal applications such as the sEMG-based gesture decoding addressed in this work [16].

First, the round-trip transmission of data and commands to and from remote servers introduces latency that is incompatible with the sub-50 ms response times required for natural, real-time prosthetic and human–machine interaction. Second, cloud inference presupposes reliable network connectivity, an assumption that fails for wearable and assistive devices intended to operate continuously and in arbitrary environments. Third, the transmission of raw

physiological signals over a network exposes sensitive medical data to security vulnerabilities and privacy violations, a concern that is especially pronounced in healthcare applications. Finally, the Cloud ML paradigm necessitates the development and maintenance of potentially costly server infrastructure, undermining the low-cost, self-contained operation that makes wearable assistive technology widely accessible.

These limitations motivate the relocation of inference from the cloud to the device itself, and served as the principal catalysts for research into Edge ML, which seeks to optimise ML frameworks for execution on mobile edge devices and MCUs. TinyML constitutes a subfield of Edge ML concerned specifically with deploying deep learning models on low-cost, highly resource-constrained MCUs through the joint optimisation of ML kernels and underlying hardware to satisfy the prevailing constraints.

1.2.4. Applications of TinyML

Since its emergence, TinyML has gained considerable traction and been adopted across numerous sectors that previously depended on Cloud ML.

The initial most well-known application of TinyML was simple wake-word detection for Amazon, Apple and Google [18]. Since then, TinyML applications have grown significantly. In the healthcare domain, this includes extensive applications in wearable technology for low-cost health monitoring, which demands the reliable and secure processing of sensitive personal data, alongside the rapid and timely detection of critical health abnormalities and emergency conditions [16, 17].

Furthermore, owing to the proximity of MCUs to sensory data and their capacity to respond with minimal latency, TinyML is well suited to gesture-based human–machine interaction (HMI). Such applications range from enabling natural and cost-effective real-time interaction within AR/VR environments to more consequential uses, such as facilitating more intuitive control of prosthetic limbs for individuals with disabilities and amputations [17].

2. State of The Art (SoTA)

It is generally difficult to assemble a clear SoTA metric given the literature sparsity on Time-Series processing especially on specific types like EMG. So in order to identify a clear SoTA body, we have to look for other studies on similar Time-Series data structures and other Bioelectric signals ML based processing on microcontrollers.

2.0.1. ECG Classification using TinyML

Owing to the scarcity of prior work on sEMG processing under TinyML constraints, we extend our review to the classification of the electrocardiogram (ECG), a bioelectric signal that shares many characteristics with the EMG and whose embedded deployment has been studied more extensively.

In the context of ECG arrhythmia detection, the study conducted by Faraone et al. [19] employs a convolutional–recurrent architecture closely related to our own. The model comprises seven one-dimensional convolutional layers with a kernel size of five, each followed by an average-pooling layer of size and stride two. The convolutional stage produces a 128-dimensional feature tensor per window, which is subsequently processed by a Gated Recurrent Unit (GRU) with 64 hidden units serving as the recurrent module.

Notably, the implementation adopts a mixed-precision quantization scheme: the convolutional, pooling, and dense layers are quantized to 8-bit fixed point (Q2.5 format), whereas the GRU gates and internal states are retained at 16-bit precision (Q2.13 format) to preserve the fidelity of the recurrent computation [19]. The model is deployed on the nRF52832 System-on-Chip from Nordic Semiconductor, an ARM Cortex-M4 platform, using the CMSIS-NN kernel library.

The network was trained and evaluated on a dataset of 8,528 single-lead ECG recordings sampled at 300 Hz with durations between 9 and 60 seconds. On this four-class classification task, the full-precision model attains a test-set accuracy of 86.1%, which decreases only marginally to 85.4% following fixed-point quantization, with the overall F1 score falling from 0.80 to 0.784. The weights, biases, and lookup tables occupy 195.6 KB of storage, while the complete output binary (including the CMSIS-DSP and CMSIS-NN routines) amounts to approximately 210 KB.

Layer	Output shape	Parameter count
Input	(N_w, 256, 1)	-
Conv1	(N_w, 128, 8)	48
Conv2	(N_w, 64, 16)	656
Conv3	(N_w, 32, 32)	2592
Conv4	(N_w, 16, 64)	10,304
Conv5	(N_w, 8, 64)	20,544
Conv6	(N_w, 4, 128)	41,088
Conv7	(N_w, 2, 128)	82,048
Global Average Pooling	(N_w, 128)	0
GRU	(64)	37,056
Dense+Softmax	(4)	260

Figure 2.1.: Convolutional–recurrent architecture employed by Faraone et al. [19].

Despite presenting a promising embedded implementation, the approach is subject to several limitations that are directly relevant to our work. First, it operates on comparatively short inputs, processing windows of only 256 samples and a recurrent sequence of just 128 timesteps, which constrains the temporal context available to the model. Second, the recurrent complexity is deliberately curtailed: the authors substitute the LSTM of the original architecture with a less expressive GRU specifically to reduce the memory footprint and operation count. Finally, and most significantly, the recurrent component could not be executed using a general-purpose inference engine; the authors report that TensorFlow Lite for Microcontrollers did not support recurrent operators at the time, necessitating a hand-written CMSIS-NN implementation.

A second representative study is the work of Kim et al. [20], which develops a TinyML-based classifier for an ECG monitoring embedded system (TinyCES). In contrast to the preceding work, the authors adopt a purely convolutional–fully-connected architecture for the binary classification of normal and abnormal ECG segments, comprising two convolutional layers (with 8 and 16 filters of size 4×1, respectively), each followed by max-pooling and dropout, and terminating in two dense layers.

The model is implemented using TensorFlow Lite for Microcontrollers and deployed on an Arduino Nano 33 Bluetooth Low Energy (BLE) Sense board, also based on an ARM Cortex-M4 core. Operating on an input window of 180 samples, the converted model occupies a footprint of only 27 KB and achieves a detection accuracy of approximately 97%.

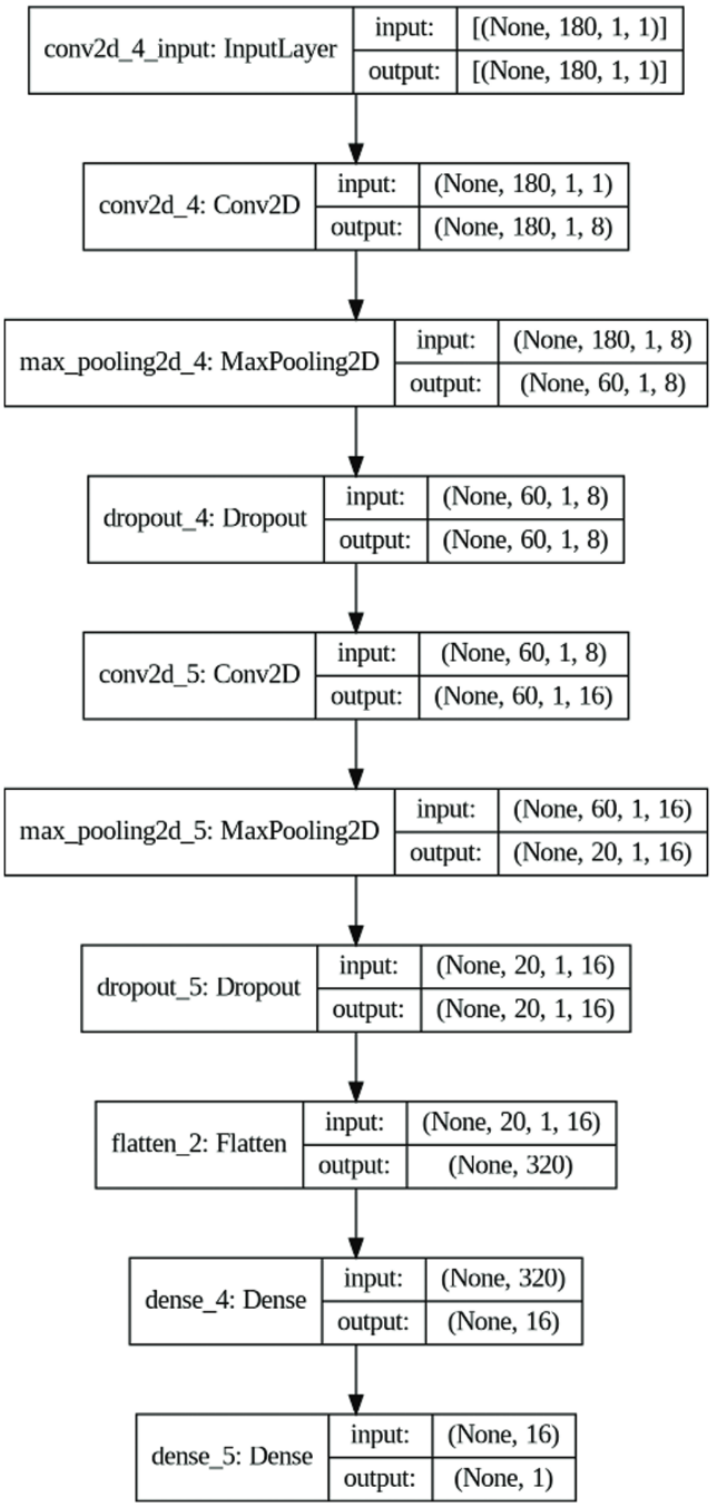


Figure 2.2.: Convolutional–fully-connected architecture employed by Kim et al. [20].

The principal limitation of this study lies in the simplicity of its architecture relative to the demands of continuous biosignal regression. The model contains no recurrent component whatsoever, relying solely on convolutional feature extraction followed by dense classification, and it therefore forgoes any explicit modelling of temporal dependencies across the

signal. Furthermore, the comparatively small input window of 180 samples and the reduction of the task to binary classification yield a markedly smaller and less complex model than is required for the multi-channel, multi-output gesture regression addressed in this thesis. Consequently, while the work convincingly demonstrates the feasibility of fully on-device ECG classification, it does not confront the memory and latency challenges posed by recurrent architectures on resource-constrained MCUs.

2.0.2. EMG Signal Processing

The closest prior work to this thesis is a pair of studies conducted by the same group at the University of Bologna, which together address the complete sEMG processing pipeline—from model design through to embedded implementation—on a multicore IoT processor. The first study targets hand-movement *classification* [14], while the second generalises the same methodology to continuous *regression* of hand kinematics [21]. As this thesis is concerned specifically with the implementation side of an analogous pipeline these works constitute the principal point of comparison.

In the classification study [14], the authors deploy TEMPONet, a dilated Temporal Convolutional Network (TCN), on the 8-core GAP8 processor. The architecture stacks three convolutional blocks, each comprising two dilated causal convolutions of kernel size 3×1 (with full padding and increasing dilation) followed by a strided convolution of kernel size 5×1 and an average pooling layer; the convolutional stage is terminated by two fully connected layers with dropout and a SoftMax operation. All layers employ ReLU activations and Batch Normalisation [14]. The model is benchmarked on the multi-session NinaPro DB6 dataset, operating on short 150 ms input windows in keeping with the 300 ms real-time consensus for hand control.

On the implementation side, the PyTorch model is deployed using the dedicated PULP-NN kernel libraries, which exploit the eight RISC-V cluster cores of the GAP8 together with their SIMD extensions. Since the GAP8 lacks a floating-point unit, inference is carried out entirely in 8-bit fixed-point arithmetic. To accommodate the tight on-chip memory, an automated tiling tool (DORY) partitions the weights and feature maps of each layer into tiles that fit the 64 kB L1 scratchpad and inserts double-buffered DMA transfers between the 512 kB L2 memory and L1 so that data movement overlaps with computation [14].

The classification model attains an inter-session accuracy of 49.6% on the full NinaPro DB6 (7.8 percentage points above the prior state of the art) and 93.7% on a 20-session dataset collected by the authors as a proxy for the final deployment. Following int8 quantisation, the network occupies a memory footprint of 460 kB and executes a single inference in 12.84 ms on the GAP8 [14].

The companion regression study [21] retains the same TCN paradigm but adapts and further optimises TEMPONet to estimate five continuous degrees of actuation, evaluating it on NinaPro DB8 (which includes two trans-radial amputees). Through channel reduction and int8 quantisation, the authors reduce the memory footprint to 70.9 kB and the inference latency to 4.76 ms on the GAP8, achieving a mean absolute error of 6.89°—marginally below the float32 LSTM baseline of the dataset [21]. Notably, the authors deliberately favour a purely convolutional model over recurrent alternatives, citing evidence that TCNs match or

exceed the accuracy of LSTMs on sEMG sequence modelling while being substantially more amenable to parallelisation and embedded deployment.

While both studies represent a compelling state of the art, their design choices differ from those of the present work in three respects that are directly relevant to our contribution. First, with regard to model complexity, both rely on a feature-extraction-only convolutional backbone and explicitly avoid any recurrent component; by contrast, the model deployed in this thesis combines spatial and temporal convolutions, residual concatenation, batch normalisation and ELU activations, a downsampling stage, a two-layer LSTM, and a dedicated regression head. The presence of a genuine recurrent module is significant, as recurrent layers are markedly more difficult to deploy on resource-constrained hardware—precisely the implementation challenge this thesis addresses. Second, with regard to the hardware platform, both works target the GAP8, an 8-core parallel processor with ISA extensions purpose-built for embedded neural-network inference, whereas this work targets the single-core ESP32-S3, a substantially more constrained but far cheaper and more widely available commodity microcontroller. Third, and most consequentially, their deployment depends on a bespoke, platform-specific toolchain (PULP-NN kernels with DORY-based tiling) tightly coupled to the GAP8 architecture, rather than on a portable, industry-standard inference framework. The present work instead builds upon LiteRT with a segmented, mixed-precision model pipeline, trading a degree of raw efficiency for portability and reproducibility on commodity hardware.

3. Materials and Methods

3.1. Dataset Description and Preprocessing

3.1.1. NinaPro DB2 Dataset

The Non-Invasive Adaptive Hand Prosthetics (NinaPro) database is the largest publicly available multi-modal benchmark assembled to support machine-learning research on human, robotic, and myoelectric hand prostheses. The database comprises ten datasets totalling more than 180 data acquisitions from both intact subjects and transradial amputees, spanning electromyographic, kinematic, inertial, clinical, neurocognitive, and eye–hand coordination modalities. Each acquisition is obtained by simultaneously recording surface electromyography (sEMG) from the forearm together with the kinematics of the hand and wrist while the subject performs a predefined set of movements and postures [22].

In this work we employ the second NinaPro database (DB2), which contains recordings from 40 intact subjects performing 49 distinct gestures, each repeated six times. The sEMG signals were acquired using twelve Delsys Trigno wireless electrodes, and the corresponding hand kinematics were captured using a CyberGlove II data glove; the dataset additionally provides inertial and force modalities. For each recording, DB2 supplies twelve sEMG channels, thirty-six accelerometer channels (the three-axis accelerometers integrated into the twelve electrodes), and twenty-two glove channels containing the uncalibrated outputs of the CyberGlove sensors, which are approximately proportional to the joint angles of the hand and wrist.

Channel and Gesture Selection

To match the input dimensionality of our model and the constraints of the target hardware, we restrict the data to a subset of channels and gestures. Of the twelve available sEMG channels, we retain only the first eight, which correspond to electrodes equally spaced around the forearm at the level of the radio-humeral joint. On the output side, the regression target is reduced to two glove channels, selected to capture the degrees of freedom most relevant to the gestures under study: one channel associated with finger abduction and fist closure, and one associated with wrist extension and flexion. For each gesture, the glove channel that is not actuated by that movement is set to zero, so that the model is trained to predict only the degree of freedom genuinely exercised by the gesture.

From the full set of 49 gestures we select five: abduction of all fingers, fingers flexed together into a fist, wrist flexion, wrist extension, and wrist extension with a closed hand; the remaining gestures are discarded. This yields a focused regression task over a small, well-characterized set of movements suited to deployment on a resource-constrained device.

Preprocessing

Each trial is band-pass filtered using a fourth-order Butterworth filter with a passband of 5–450 Hz, attenuating frequency components outside this range. Every trial is epoched to a fixed length of five seconds, corresponding to 10,000 samples at the DB2 sampling rate. The epoched recordings are then segmented into non-overlapping windows; that is, the windowing hop is set equal to the window length, so that consecutive windows tile each trial without overlap. The window length itself is treated as an experimental parameter, allowing its effect on both predictive accuracy and on-device latency to be studied systematically.

3.2. Deep Learning Model Architecture

The model architecture used in this work was developed by Koellner et al. at the Institute for Signal Processing and System Theory, University of Stuttgart, and was trained on the NinaPro DB2 dataset. It is a fully convolutional–recurrent regression network that maps a windowed, multi-channel sEMG input of shape (B, C, T)—where B is the batch size, C the number of input channels, and T the window length—to a continuous estimate $\hat{y}$. For the present study, the input comprises $C = 8$ EMG channels. The network is organized into four functional stages: a convolutional feature extractor (input block), a convolutional downsampling block, a recurrent block, and a regression head, as illustrated in Figure 3.1.

Input block (feature extraction). The input block performs feature learning along the temporal and spatial (inter-channel) axes in sequence. The windowed signal is first expanded to $(B, 1, C, T)$ and passed through a temporal two-dimensional convolution with 48 output channels, a kernel of size $(1, k)$ and same padding, followed by batch normalization. A spatial convolution with a kernel of size $(C, 1)$ and channel-wise grouping then mixes information across the input channels, after which the tensor is squeezed back to $(B, 48, T)$ and subjected to a further batch normalization, an ELU activation, and dropout with a rate of 0.5. The output of the input block is concatenated with the original input along the channel dimension, yielding a feature representation of shape $(B, 48+C, T)$. This concatenation acts as a residual-style skip connection that preserves the raw input alongside the learned features.

Downsampling block. The concatenated feature map is downsampled along the temporal axis by a strided, grouped one-dimensional convolution with $48 + C$ output channels, a kernel of size 9, same padding, and a stride equal to the downsampling factor of 2. This is followed by batch normalization, an ELU activation, and dropout with a rate of 0.5, producing a tensor of shape $(B, 48 + C, T/2)$ in which the temporal resolution has been halved.

Recurrent block. Temporal dependencies are modelled by a two-layer unidirectional LSTM with a hidden size of $48 + C = 56$, which preserves the channel dimension of its input and outputs a sequence of shape $(B, 48 + C, T/2)$.

Regression head. The final stage is a fully connected regressor that maps the LSTM output to the target. It consists of a linear layer projecting from $48+C$ to a hidden width of 11, a tanh activation, and a second linear layer projecting from 11 to the C output dimensions, yielding the prediction $\hat{y}$ of shape $(B, C, T/2)$.

Model Configurations

The architecture is highly configurable. To investigate the effect of the receptive field of the temporal convolution on both accuracy and on-device performance, we evaluated two configurations that differ principally in the dilation and kernel size of the input block and in the depth of the regression head; all remaining hyperparameters are held constant. Both configurations use a dropout rate of 0.5 throughout, a downsampling stride of 2, and a two-layer LSTM with a hidden size of 56.

Model A[1] employs an undilated temporal convolution with a wide kernel: the input block uses a kernel size of 51, 48 output channels, and a dilation of 1, and the regression head comprises four layers. Model B instead employs a dilated temporal convolution with a narrower kernel: the input block uses a kernel size of 17, 48 output channels, and a dilation of 3, with a three-layer regression head. The configurations are summarized in Table 3.1.

Table 3.1.: Hyperparameter configurations of the two evaluated models.

Block	Hyperparameter	Model A	Model B
Input	Output channels	48	48
	Kernel size	51	17
	Dilation	1	3
	Dropout	0.5	0.5
Downsample	Kernel size	9	9
	Stride	2	2
	Dropout	0.5	0.5
LSTM	Layers	2	2
	Hidden size	56	56
Regressor	Layers	4	3

[1] Designate as baseline / dilated as appropriate.

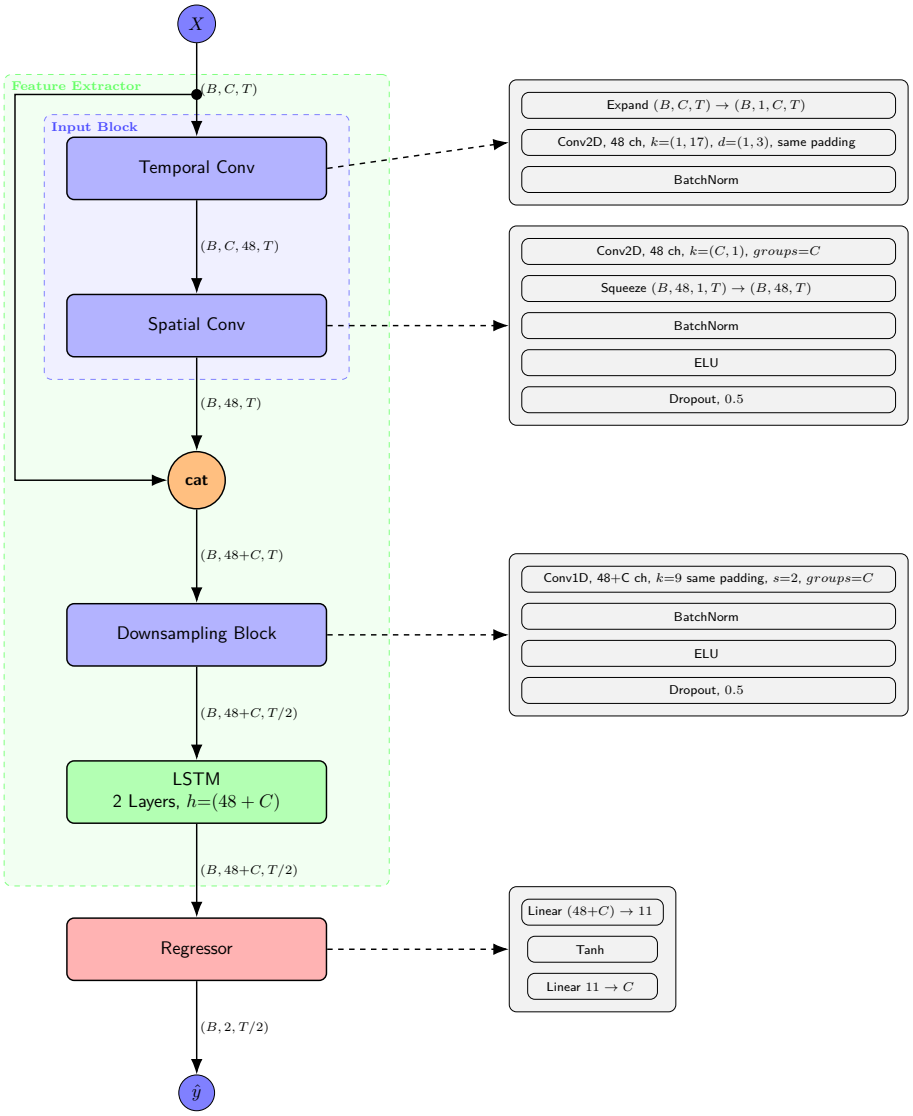


Figure 3.1.: Deep Learning Architecture used for EMG regression by Koellner et al

3.3. Hardware

3.3.1. ESP32-S3 Specifications

All experiments in this work were conducted on an ESP32-S3-based development board (ESP32-S3 UNO) equipped with 16 MB of external flash and 8 MB of external PSRAM (pseudo-static RAM). The ESP32-S3 was selected because it is one of the few low-cost microcontrollers whose architecture incorporates dedicated hardware acceleration for the integer arithmetic that dominates quantized neural-network inference. This subsection summarizes the characteristics of the platform that are most relevant to on-device machine learning and to the single-instruction-multiple-data (SIMD) execution exploited by the ESP-NN backend. All figures are taken from the official *ESP32-S3 Technical Reference Manual* [23].

Processor and Memory Architecture

The ESP32-S3 is a dual-core system built around two Harvard-architecture Tensilica Xtensa® LX7 CPUs, each capable of operating at a clock frequency of up to 240 MHz. On-chip memory comprises 384 KB of internal ROM and 512 KB of internal SRAM, the latter accessible by the CPU typically within a single clock cycle. This relatively small fast-memory budget is a defining constraint of the platform: the complete set of model parameters, the interpreter's tensor arena, and all runtime buffers must either fit within the internal SRAM or be placed in the slower external memory.

To accommodate larger models, the ESP32-S3 supports external flash and external RAM through its SPI interfaces (including the Octal SPI and OPI modes used by high-bandwidth PSRAM), addressing up to 1 GB of each. The CPU does not access external memory directly; instead, the external address space is mapped into the CPU address space through a cache. Up to 32 MB of the data-bus address space can be mapped to external RAM in individual 64 KB blocks. This indirection is significant for inference latency: data resident in the 8 MB PSRAM of our board is reached through the cache rather than at SRAM speed, so any working set (notably a large activation arena) that spills out of internal SRAM into PSRAM incurs additional access latency on cache misses.

Memory access on the ESP32-S3 is mediated by a dual-core-shared instruction cache (ICache) and data cache (DCache). The ICache can be configured to 16 KB or 32 KB with a block size of 16 B or 32 B, while the DCache can be configured to 32 KB or 64 KB with a block size of 16 B, 32 B, or 64 B. The cache controller fetches from external memory on a miss and supports write-back, clean, invalidate, and preload operations. Because cache misses against PSRAM are far more expensive than internal-SRAM accesses, the locality of the inference working set has a direct effect on measured latency—a consideration that motivates keeping the tensor arena small enough to remain in internal memory wherever possible.

Processor Instruction Extensions (PIE) and SIMD

The feature most relevant to this work is the ESP32-S3's Processor Instruction Extensions (PIE), a vector instruction set defined through the Tensilica Instruction Extension (TIE) language and added specifically to accelerate AI and digital-signal-processing workloads. The PIE follows the SIMD paradigm, applying a single instruction across multiple data elements packed into wide vector registers, and supports 8-bit, 16-bit, and 32-bit integer vector operations.

The unit provides a bank of eight 128-bit general-purpose vector registers (denoted QR). Each register can be interpreted as sixteen 8-bit, eight 16-bit, or four 32-bit lanes, depending on the operation. The arithmetic logic unit contains sixteen 8-bit multipliers and eight 16-bit multipliers, enabling up to sixteen multiply-and-accumulate operations to be issued in a single 8-bit instruction. Results accumulate into dedicated wide registers—a pair of 160-bit accumulators (QACC_H and QACC_L) and a 40-bit accumulator (ACCX)—which preserve the precision of summed integer products before the result is requantized and written back. Arithmetic instructions such as multiplication, shifting, and accumulation can transfer data while computing, and the extension supports saturating arithmetic and non-aligned 128-bit vector loads.

This hardware is directly responsible for the performance characteristics reported in Chapter **??**. The wide multiply-accumulate datapath maps naturally onto the dominant operations of the inference pipeline—the matrix multiplications underlying the fully connected and LSTM layers, and the convolutions of the feature-extraction stages—but only when those operations are expressed as 8-bit integer arithmetic. The eight 8-bit lanes per multiplier group and the abundance of 8-bit multipliers explain why the ESP-NN kernels are optimized exclusively for `int8` operands on this platform, and why the floating-point execution paths, which cannot use the PIE multiply-accumulate units in the same way, are substantially slower. The architectural details presented here therefore underpin the large `int8`-versus-floating-point latency gap observed in our measurements.

3.4. Quantization

Normally, Deep learning pipelines rely on floating point operations and embed their parameters (i.e weights and biases) as float32s. Unfortunately, not all MCU architectures have embedded FPU (floating point units) to handle calculations in floating point notations. In addition, chips that have dedicated FPUs in their architecture, for handling float32 data, suffer from longer processing times and resource intensive operation compared to the faster integer arithmetic units [24, 25, 26].Furthermore, floating point numbers necessitate larger spaces in memory for tensor allocations (i.e inputs, outputs, model parameters and activations) [24].
For these reasons, different quantization schemes are widely adopted across TinyML literature It is also the case on many MCU platforms (like the ESP32 and ESP32s3), that optimized low-level kernels for ML ops (like Convolutions and MatMuls) are exclusively written with quantization schemes, like INT8, in mind. In fact, 8-bit quantization has been shown to reduce model sizes by a factor of 4 and produce a speedup gain of 2x-3x ,even 10x on specialized processors, compared to float Implementations[24].

On the other hand, quantization algorithms are not lossless since they map a continuous floating point representation into a discrete range. A degradation in final accuracy is always expected in quantized outputs. That's why quantization pipelines require careful fine-tuning to balance quality and performance.

3.4.1. Quantization Types and Schemes

Quantization schemes have been categorized into many section. In this work, we are going to outline schemes that are only relevant to TinyML research.

Data Types

By default, popular frameworks like PyTorch, Tensorflow and JAX utilize single-precision floating point numbers (float32s). Quantization schemes can then be categorized into the output data types that these float32s are converted to.

Float16 quantization uses half the space used by float32s while maintaining the floating point interface as float32s. However, float16 adoption in MCU architectures still remain

rare. Most MCUs don't have dedicated kernels for handling float16s and just use float32 interfaces rendering float16 optimizations completely useless. Others are unable to handle float16 data and immediately throw exceptions.

Int8, Int16, Int32 quantization are by far the most used. Most MCU architectures are optimized for handling integer operations which makes int quantization adoption easier and more natural. Low-Level kernels usually use a mix of various integer sizes in their internal Implementation, usually inputs, outputs and weights are int8 while biases use larger formats like int32s.

Asymmetric vs symmetric quantization

symmetric quantization restricts the zero-point to 0. It only computes the scale of the floating-point values as the ration between the absolute maximum value in the floating point range and the maximum value in the quantization range. Commonly used for quantizing weights and biases as[24]. The reason why it is preferred for weights is because using asymmetric quantization (with an additional zero-point value for weights) adds an additional term to each **W . x** calculation which costs additional compute time during inference [26].

$$
\begin{aligned}
x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) \\
x_Q &= \text{clamp}(-N_{levels}/2,\ N_{levels}/2-1,\ x_{int}) && \text{if signed} \\
x_Q &= \text{clamp}(0,\ N_{levels}-1,\ x_{int}) && \text{if un-signed} \\
x_{out} &= x_Q\,\Delta && \text{(de-quantization)}
\end{aligned}
\tag{3.1}
$$

Asymmetric quantization calculates an optimal new zero-point value to map the floating point zero to it. Preferred for activations.

Quantization Aware Training (QAT) vs Post Training Quantization (PTQ)

$$
\begin{aligned}
x_{int} &= \text{round}\left(\frac{x}{\Delta}\right) + z \\
x_Q &= \text{clamp}(0,\ N_{levels}-1,\ x_{int}) \\
x_{float} &= (x_Q - z)\,\Delta && \text{(de-quantization)}
\end{aligned}
\tag{3.2}
$$

Post Training Quantization (PTQ) works on quantization pre-trained DL models, trained using default floating-point numbers. The quantizing algorithm takes pre-trained weights and biases and calculates the appropriate scaling and shifting parameters. For activations, it uses a representative dataset (representing a sample of input values in floating space) and propagates it through the model to get a closer statistical model of how values fluctuate in the actual model in order to map it to the quantized range. PTQs are widely adopted in pipelines that aim at deploying normal, pre-trained models, usually not co-designed for TinyML use, on Edge device without having to re-train the model [24, 26].

Quantization Aware Training (QAT) involves having quantization in mind during the training of the actual model, with simulated quantization applied at every step of gradient

descent so that the model learns to compensate for quantization noise. The weights are maintained in floating point and updated using floating-point gradients, while a simulated quantization operation (fake quantization) emulates the effect of integer inference during the forward and backward passes. While this method can provide higher accuracies than PTQ [24, 26], it necessitates active co-design of the DL model and is not viable for quantizing all available models, since it requires retraining in the presence of simulated quantization, which demands additional training resources and time-consuming retraining.

$$\begin{aligned} w_{float} &= w_{float} - \eta \frac{\partial L}{\partial w_{out}} \cdot I_{w_{out} \in (w_{min}, w_{max})} \\ w_{out} &= \text{SimQuant}(w_{float}) \end{aligned} \tag{3.3}$$

Hybrid vs Full quantization

Hybrid quantization is defined as quantizing the inputs, weights and biases only while leaving intermediate activations as floating point. While this can improve final accuracy since a full quantization is not adopted and activations remain in their full floating point resolution, many MCU platforms and frameworks do not fully support mixed precision computation. In addition, Hybrid models produced the overhead of having to requantize activations after each neural layer which can detrimentally impact the final inference time and the needed allocation space for larger float32 intermediate values.

Full quantization involves the quantization of the entire Pipeline eliminating the need for intermittent requantization steps and produces more faster and more lightweight models since there are not intermediate floats. However, for full quantization, more overhead is needed for calculating additional quantization parameters for activation layers and , depending on the quantization scheme used, may require a set of representative data to calculate those quantization parameters.s

Quantization granularity

Quantization can be done with varying degrees of resolution at the cost of producing an increasing amount of quantization parameters and settings.

Per Tensor quantization involves calculating scaling and shifting params for each tensor involved in the pipeline. This scheme is preferred for activations [24].

Per Channel quantization involves having a set of quantization params per each channel in a tensor. This presents a more granular quantization scheme as it accounts for distribution differences between individual channels in the data at hand. This is the recommended method for stored weights [24, 26].

3.4.2. Quantization Scheme Used

In this work, we opted for a PTQ quantization scheme. In order to comply with ESP-NN backend functions, we use full int8 quantization across the entire model with int32 accumu-

lators and int32 for biases as presented here [25]. Since ESP-NN uses the same quantization schemes as TFlite runtime, we are using per tensor asymmetric quantization for inputs and activations while using symmetric per channels quantization for both weights and biases. Similar 8-bit PTQ schemes, with per channel quantization for weights and per layer quantization of activations, have shown accuracies within 2 percent of floating point networks, particularly for CNN architectures [24].

3.5. Software and Tools

3.5.1. ESP-NN Library

The ESP-NN library contains optimized NN (Neural Network) functions for various Espressif chips. They are the backend used by TensorFlow Lite Micro on esp32 chips. It supports ESP32, ESP32-S3, ESP32-P4 and ESP32-C3 chips. While it provides generic optimizations for most chips, it provide assembly optimized versions for ML dedicated chips like ESP32-S3.

The optimized ESP-NN libraries on ESP32-S3 chip are only dedicated for Int8 quantized operations. For matrix multiplications, the core arithmetic operations behind fully connected and LSTM layers, 2 functions are available for use. `esp_nn_fully_connected` which is used for per tensor weight quantization and the `esp_nn_fully_connected_per_ch` variant for per channel quantized weights.

Similar to [25], ESP-NN uses int32 accumulators for accumulating the intermediate products in matmul operations. This follows directly from the integer-arithmetic-only formulation of [25], where each real value r is represented through an affine mapping $r = S(q - Z)$ from its quantized integer q, with scale S and zero-point Z. Under this mapping, a quantized matrix product reduces to:

$$q_3^{(i,k)} = Z_3 + M \sum_{j=1}^{N} \left(q_1^{(i,j)} - Z_1\right)\left(q_2^{(j,k)} - Z_2\right), \tag{3.4}$$

where the summation is performed entirely in integers and held in a wide (int32) accumulator, exactly as ESP-NN does. The only non-integer term is the multiplier $M = S_1 S_2 / S_3$, which depends only on the fixed tensor scales and is therefore constant at inference. Since $M \in (0, 1)$, it is expressed in the fixed-point form $M = 2^{-n} M_0$ with $M_0 \in [0.5, 1)$. This is precisely how the rescaling is discretized into the kernel's two integer parameters: M_0 becomes the fixed-point multiplier `out_mult`, and the factor 2^{-n} becomes a rounding right-shift by `out_shift`. Requantization thus reduces to one integer multiply followed by a shift, keeping the matmul pipeline entirely integer-only.

3.5.2. LiteRT (TensorFlow Lite) Framework

LiteRT is the successor to the widely adopted TensorFlow Lite pipeline. Developed by Google, TensorFlow Lite was designed to streamline TinyML workflows by providing a means of converting pre-trained TensorFlow models into a flatbuffer representation (the

`.tflite` format). This flatbuffer is subsequently read by the TensorFlow Lite Micro interpreter on the target device to perform inference. A principal strength of the framework is that it delegates execution to the optimized neural-network kernels native to the host microcontroller—such as ESP-NN for Espressif devices and CMSIS-NN for ARM-based devices—thereby achieving near-native performance without requiring the developer to implement the model from scratch using low-level kernels.

The underlying development pipeline of TensorFlow Lite Micro (TFLM) comprises two distinct stages, namely an offline conversion stage performed on a host machine and an on-device inference stage executed on the target microcontroller [18]. In the offline stage, a model trained within the TensorFlow environment is processed by the TensorFlow Lite converter, which applies optimisations such as quantisation and the folding of constant expressions (e.g. batch normalisation) before serialising the result into a portable FlatBuffer file (`.tflite`). The FlatBuffer format is well suited to embedded targets, as its memory-mapped representation requires no unpacking at run time and can be readily embedded into the application binary as a C source array, obviating the need for a file system on devices that lack one [18].

In the on-device stage, rather than generating native code from the model, TFLM adopts an interpreter-based design in which a single, static body of execution code reads the FlatBuffer at run time to determine which operators to invoke and from where to draw their parameters [18]. This approach favours portability and field-updatability over the marginal performance gains of code generation, and incurs negligible overhead in practice, since the long-running neural-network kernels dominate execution time and amortise the cost of interpretation [18]. At initialisation, the application supplies the interpreter with the model, an operator resolver (`OpResolver`) that maps the operators present in the model to their implementations whilst minimising binary size, and a contiguous, fixed-size memory region termed the *arena*. The interpreter plans and allocates all required buffers from this arena during the initialisation phase and performs no further dynamic allocation during inference, thereby avoiding the heap fragmentation that would otherwise jeopardise long-running embedded applications [18]. It is precisely at the operator level of this interpreter that the aforementioned vendor-optimised kernel libraries are substituted for the portable reference implementations, allowing hardware-specific acceleration without modification to the surrounding framework [18].

The principal limitation of TensorFlow Lite was its tight coupling to the TensorFlow ecosystem, which required PyTorch models to be translated into TensorFlow before conversion. This translation was not straightforward: it entailed first exporting the PyTorch model to the ONNX interchange format and then converting the ONNX model to TensorFlow. In our experience, this multi-stage pipeline incurred substantial conversion overhead and, in many cases, failed to complete owing to discrepancies between the PyTorch and TensorFlow model representations—for example, differences in tensor memory layout (NCHW versus NHWC) and in the expected index of the batch dimension when exporting LSTM layers.

Released in 2024, LiteRT removes this dependency on the TensorFlow ecosystem, repositioning the framework as a cross-framework solution that no longer requires intermediate conversion formats, while retaining the `.tflite` file format for backward compatibility. LiteRT additionally exposes a range of configurable quantization schemes, including Float16, hybrid, and full Int8 quantization.

Modular Architecture

To address these issues, we adopted a modular deployment strategy. Rather than exporting the entire network as a single flatbuffer with one long and costly graph, we partitioned the model at carefully chosen points so as to eliminate redundant operations and to permit independent selection of quantization scheme and numerical precision for each part, with the joint objectives of minimizing mean squared error and reducing the memory footprint. After experimenting with the placement of these partitions, we settled on three split points that divide the model into four stages.

The partitioning was performed non-destructively: each stage was reconstructed as a standalone model from the trained network, with the corresponding parameters copied into the appropriate stage. The resulting four stages comprise two convolutional models for the feature-extraction operations (the temporal and spatial convolutions of the input block forming the first stage, and the downsampling convolution forming the second), a single instance of the two-layer LSTM cell as the third stage, and the regression head as the fourth.

A key element of this strategy concerns the export of the recurrent stage. Rather than unrolling and exporting the LSTM over the entire window—which had produced the prohibitive footprint and initialization cost described above—the LSTM is exported as a single block spanning only a small, fixed number of timesteps (one or five in our experiments) and is then invoked repeatedly within a loop on the device, together with the regression head, to process the full sequence. Because only one short LSTM block resides in memory at any time and is reused across iterations, rather than the complete set of unrolled timesteps being materialized as distinct cells, this windowed export reduces the memory overhead of the recurrent stage considerably. The number of timesteps processed per block (the LSTM block step size) therefore becomes a deployment parameter that trades recurrent loop overhead against the working memory of the stage. Taken together, these measures reduced the memory footprint from over 1.5 MB to approximately 260 KB for the complete model. Crucially, the modular decomposition also enabled mixed-precision deployment, allowing accurate floating-point LSTM stages to operate alongside optimized Int8-quantized convolutional stages.

Graph-Level Transformations for Quantization

Two further non-destructive transformations of the model graph were necessary to obtain correct and efficient quantized stages.

First, the batch-normalization layers were folded manually into the weights of the preceding convolutions. The LiteRT converter was unable to fuse the `BatchNorm1d` layers on its own; instead, it realized each normalization as a separate pair of multiply and add operations in the graph. Beyond the redundant computation this introduced, it aggravated the quantization problem, since both operations were placed inside a floating-point quantize/dequantize island. We therefore folded the batch-normalization parameters back into the corresponding convolution weights and biases analytically and removed the layers from the graph, eliminating both the spurious operations and the associated precision conversions.

Second, the one-dimensional convolutions were reformulated as two-dimensional convolutions. The LiteRT and TensorFlow Lite Micro toolchain does not quantize `Conv1D` operations

by default; expressing the same computation as a `Conv2D` with a singleton spatial dimension was required for the convolutional stages to be quantized correctly.

Experimental Workflow

The experimental workflow proceeds as follows. A configuration-driven system accepts a list of model configurations—specifying parameters such as the window length and the LSTM block step size—and batch-converts each configuration to the LiteRT format, exporting eight models per configuration: four in floating point and four Int8-quantized, corresponding to the four pipeline stages. The exported models are then packed into a single contiguous binary file and flashed to a dedicated partition of the microcontroller's flash memory. The complete input trial is exported alongside it, together with the reference output produced by the original PyTorch model, against which the mean squared error is computed. On the ESP32-S3, a continuous loop iterates over all configurations to generate the final results.

4. Results

4.1. Comparison of ML Ops Performance on the ESP32-S3 using Random Weights under different Quantization Schemes

In order to evaluate general Model perfromance on the esp32s3, we had to create our own small benchmark to evaluate the behavior of different data layers and how they react to different parameter changes and to int8 quantization used.

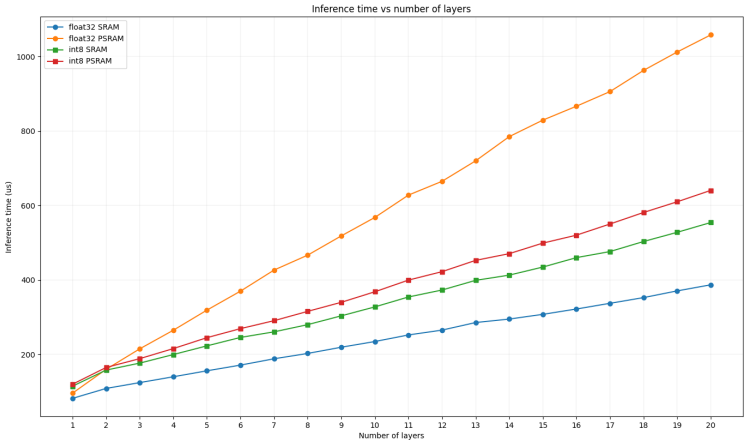


Figure 4.1.: Figure shows latency of dense layers against the amount of layers used in a geometric decay.

The above graph shows how the number of layers impacts the final latency for dense layers, where a geometric falloff determines the number of units at each stage, decaying from 50 down to 2. Across all four configurations latency grows approximately linearly with the number of layers. The effect of int8 quantization, however, depends strongly on the memory in which the model resides. On PSRAM, int8 quantization reduces latency relative to float32 (e.g. $\sim 640\,\mu s$ vs. $\sim 1060\,\mu s$ at 20 layers), as the four-fold reduction in weight size lowers traffic over the slow external bus on this memory-bound workload. On SRAM the trend reverses: int8 is consistently slower than float32 (e.g. $\sim 555\,\mu s$ vs. $\sim 386\,\mu s$ at 20 layers), because the workload becomes compute-bound and the ESP32-S3's hardware floating-point unit executes each float multiply-accumulate in a single instruction, whereas the int8 path incurs additional per-channel requantization overhead. The fastest configuration overall is float32 on SRAM, and the slowest is float32 on PSRAM.

Table 4.1.: Conv2D layers performance on the ESP32-S3.

			PSRAM				No PSRAM			
			Float32		Int8		Float32		Int8	
	Input	Architecture	Inf (μs)	MSE	Inf (μs)	MSE	Inf (μs)	MSE	Inf (μs)	MSE
Layers	(1, 2, 50)	Conv(in=1, out=10, L=1, k=(1, 5))	1813	0	1198	0.0003	1574	0	1191	0.0003
	(1, 2, 50)	Conv(in=1, out=10, L=2, k=(1, 5))	2933	0	1768	0.0005	2908	0	1755	0.0005
	(1, 2, 50)	Conv(in=1, out=10, L=3, k=(1, 5))	2807	0	1701	0.0036	2785	0	1680	0.0036
	(1, 2, 50)	Conv(in=1, out=10, L=4, k=(1, 5))	2464	0	1556	0.0022	2453	0	1548	0.0022
Input Size	(1, 10, 10)	Conv(in=1, out=10, L=1, k=(2, 2))	1636	0	1133	0.0002	1472	0	1125	0.0002
	(1, 20, 20)	Conv(in=1, out=10, L=1, k=(2, 2))	7209	0	3817	0.0004	5597	0	3477	0.0004
	(1, 30, 30)	Conv(in=1, out=10, L=1, k=(2, 2))	23227	0	8489	0.0006	12642	0	7500	0.0006
	(1, 40, 40)	Conv(in=1, out=10, L=1, k=(2, 2))	50534	0	15464	0.0005	22606	0	13179	0.0005
Kernel Size	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 5))	1832	0	1217	0.0003	1628	0	1219	0.0003
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 10))	2338	0	1308	0.0002	2260	0	1305	0.0002
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 20))	3386	0	1458	0.0002	3297	0	1436	0.0002
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 30))	4118	0	1526	0.0002	4014	0	1514	0.0002
Output Ch	(1, 1, 100)	Conv(in=1, out=1, L=1, k=(1, 5))	389	0	448	0.0002	386	0	445	0.0002
	(1, 1, 100)	Conv(in=1, out=20, L=1, k=(1, 5))	3439	0	1433	0.0003	2943	0	1353	0.0003
	(1, 1, 100)	Conv(in=1, out=30, L=1, k=(1, 5))	5077	0	2694	0.0003	4253	0	2469	0.0003

Table 4.2.: Conv2D layers performance on the ESP32-S3.

			PSRAM				No PSRAM			
			Float32		Int8		Float32		Int8	
	Input	Architecture	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)
Layers	(1, 2, 50)	Conv(in=1, out=10, L=1, k=(1, 5))	2.3	7.9	2.6	2.6	2.3	7.9	2.6	2.6
	(1, 2, 50)	Conv(in=1, out=10, L=2, k=(1, 5))	3.6	4.0	3.8	3.2	3.6	4.0	3.8	3.2
	(1, 2, 50)	Conv(in=1, out=10, L=3, k=(1, 5))	4.9	2.9	4.8	3.4	4.9	2.9	4.8	3.4
	(1, 2, 50)	Conv(in=1, out=10, L=4, k=(1, 5))	6.0	3.0	5.8	3.5	6.0	3.0	5.8	3.5
Input Size	(1, 10, 10)	Conv(in=1, out=10, L=1, k=(2, 2))	2.3	7.1	2.6	2.4	2.3	7.1	2.6	2.4
	(1, 20, 20)	Conv(in=1, out=10, L=1, k=(2, 2))	2.3	29.0	2.6	7.8	2.3	28.9	2.6	7.8
	(1, 30, 30)	Conv(in=1, out=10, L=1, k=(2, 2))	2.3	66.5	2.6	17.2	2.3	66.4	2.6	17.2
	(1, 40, 40)	Conv(in=1, out=10, L=1, k=(2, 2))	2.3	119.6	2.6	30.5	2.3	119.6	2.6	30.5
Kernel Size	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 5))	2.3	8.2	2.6	2.6	2.3	8.2	2.6	2.6
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 10))	2.5	7.9	2.6	2.6	2.5	7.9	2.6	2.6
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 20))	2.9	7.1	2.7	2.4	2.9	7.1	2.7	2.4
	(1, 1, 100)	Conv(in=1, out=10, L=1, k=(1, 30))	3.3	6.3	2.8	2.2	3.3	6.3	2.8	2.2
Output Ch	(1, 1, 100)	Conv(in=1, out=1, L=1, k=(1, 5))	1.7	1.4	1.9	0.9	1.7	1.4	1.9	0.9
	(1, 1, 100)	Conv(in=1, out=20, L=1, k=(1, 5))	2.5	15.8	2.9	4.6	2.5	15.8	2.9	4.6
	(1, 1, 100)	Conv(in=1, out=30, L=1, k=(1, 5))	2.8	23.4	3.2	6.6	2.8	23.4	3.2	6.5

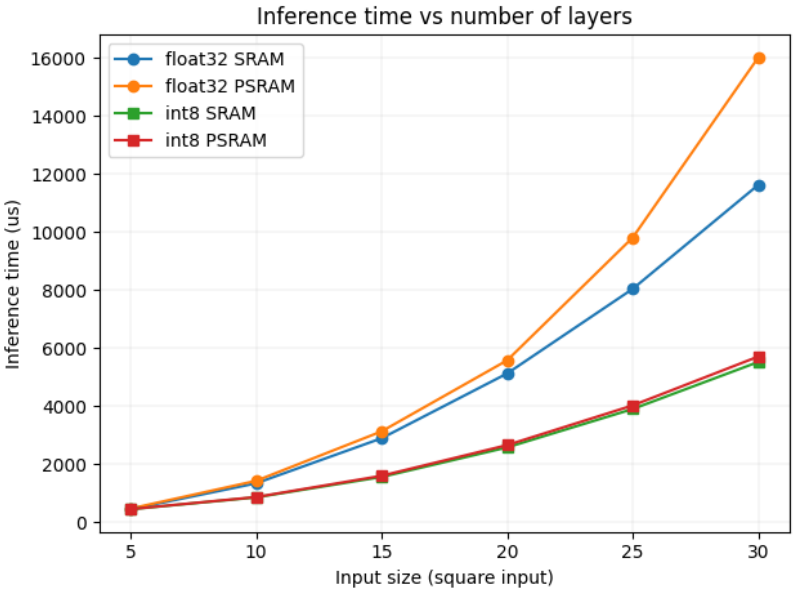
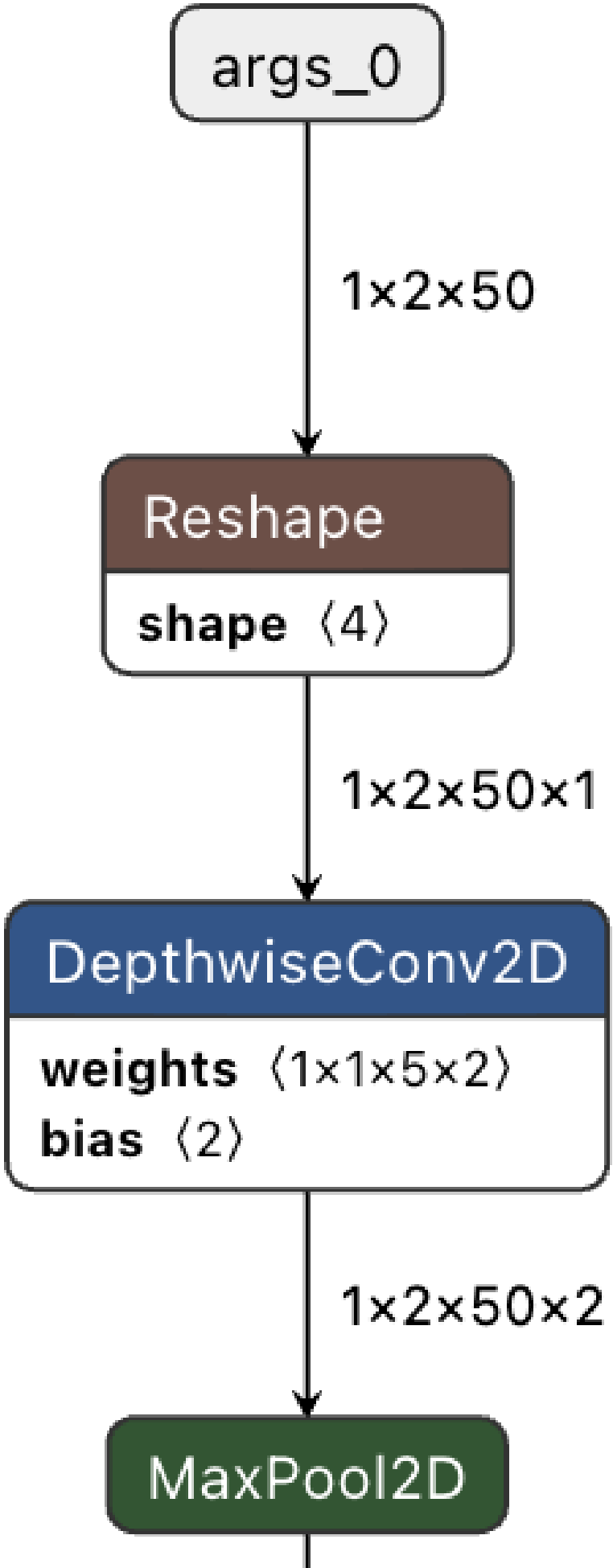


Figure 4.2.: Figure shows latency of Conv2D layers against Input Size.


args_0
1×2×50
Reshape
shape ⟨4⟩
1×2×50×1
DepthwiseConv2D
weights ⟨1×1×5×2⟩
bias ⟨2⟩
1×2×50×2
MaxPool2D

Across all Conv2D configurations, inference latency on internal SRAM is consistently lower than on PSRAM; for the $(1, 40, 40)$ input the float32 latency falls from $50534\,\mu s$ to $22606\,\mu s$ when the model is executed from SRAM. Int8 quantization yields a further substantial reduction, lowering the same configuration from $50534\,\mu s$ (float32, PSRAM) to $15464\,\mu s$ (int8, PSRAM), a 3.27× speedup, at a negligible accuracy cost (MSE ≤ 0.0006 throughout). The benefit of quantization is workload-dependent: for the smallest layer (a single output channel) int8 is marginally slower than float32 ($448\,\mu s$ vs. $389\,\mu s$), whereas its relative advantage widens as the layer grows, reaching the 3.27× factor noted above.

Latency increases monotonically with input size, kernel size, and the number of output channels, as expected from the corresponding growth in the convolution workload. The layer-count sweep is the exception: because the architecture employs "same" convolutions with a MaxPool2D stage after each layer and assigns intermediate channel counts via geometric decay, each additional layer both halves the spatial extent of the feature map (Fig. 4.3, $50 \rightarrow 25 \rightarrow 12 \rightarrow 6$) and alters the distribution of intermediate channel widths. As a result, the per-layer cost falls rapidly with depth, and total latency is not monotonic in the number of layers; the four-layer model ($1556\,\mu s$, int8 PSRAM) is faster than the three-layer model ($1701\,\mu s$) because its intermediate channel allocation yields less work despite the additional stage.

The memory footprint (Table 4.2) exhibits a clear separation between model size and arena (working-memory) requirements. Model size is largely insensitive to quantization for these small networks; because the parameter counts are modest, the per-tensor quantization metadata roughly offsets the four-fold reduction in weight precision, so int8 models are in fact marginally larger than their float32 counterparts for the smallest configurations (e.g. 2.3 KB $\rightarrow$ 2.6 KB for the single-layer k=(1, 5) model), and only become smaller once the weight tensors dominate (e.g. 3.3 KB $\rightarrow$ 2.8 KB at k=(1, 30)). The arena, by contrast, is dominated by activation tensors and is where quantization delivers its principal benefit: for the $(1, 40, 40)$ input the arena falls from 119.6 KB (float32) to 30.5 KB (int8), a 3.9× reduction. Arena size grows steeply with the activation volume, increasing with input size (7.1 $\rightarrow$ 119.6 KB for float32 as the input grows from 10×10 to 40×40) and with the number of output channels (1.4 $\rightarrow$ 23.4 KB from one to thirty channels), while remaining identical between the PSRAM and No-PSRAM configurations, confirming that memory placement affects latency but not footprint.

Table 4.3.: LSTM layers performance on the ESP32-S3.

			PSRAM				No PSRAM			
			Float32		Int8		Float32		Int8	
	Input	Architecture	Inf (μs)	MSE	Inf (μs)	MSE	Inf (μs)	MSE	Inf (μs)	MSE
Time-Steps	(1, 1, 25)	LSTM(25, 25, 1)	1417	0	1523	0	896	0	1432	0
	(5, 1, 25)	LSTM(25, 25, 1)	3736	0	5731	0	2495	0	4894	0
	(10, 1, 25)	LSTM(25, 25, 1)	6513	0	11234	0	4409	0	9431	0
	(15, 1, 25)	LSTM(25, 25, 1)	9542	0	17538	0.0036	6460	0	14733	0.0036
	(20, 1, 25)	LSTM(25, 25, 1)	12346	0	23874	0.0030	——	——	——	——
Hidden Size	(1, 1, 50)	LSTM(50, 50, 1)	4111	0	2836	0	1455	0	2268	0
	(1, 1, 100)	LSTM(100, 100, 1)	11961	0	8874	0	——	——	——	——
	(1, 1, 150)	LSTM(150, 150, 1)	24933	0	17038	0	——	——	——	——
	(1, 1, 200)	LSTM(200, 200, 1)	42847	0	27723	0	——	——	——	——
Layers	(1, 1, 25)	LSTM(25, 25, 2)	3225	0	3312	0.0045	1595	0	2870	0.0045
	(1, 1, 25)	LSTM(25, 25, 3)	4353	0	4764	0.0029	2054	0	3917	0.0029
	(1, 1, 25)	LSTM(25, 25, 4)	5540	0	6261	0.0050	2519	0	5020	0.0050
	(1, 1, 25)	LSTM(25, 25, 5)	6677	0	7776	0.0014	2968	0	6170	0.0014
	(1, 1, 25)	LSTM(25, 25, 6)	7867	0	9303	0.0005	——	——	——	——

Table 4.4.: LSTM layers performance on the ESP32-S3.

			PSRAM				No PSRAM			
			Float32		Int8		Float32		Int8	
	Input	Architecture	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)	Size (KB)	Arena (KB)
Time-Steps	(1, 1, 25)	LSTM(25, 25, 1)	25.8	3.2	16.0	5.5	25.8	3.2	16.0	5.5
	(5, 1, 25)	LSTM(25, 25, 1)	41.4	9.2	43.5	21.1	41.4	9.2	43.5	21.1
	(10, 1, 25)	LSTM(25, 25, 1)	60.9	16.5	77.6	41.3	60.9	16.5	77.6	41.3
	(15, 1, 25)	LSTM(25, 25, 1)	81.5	24.4	113.2	63.2	81.5	24.4	113.2	63.2
	(20, 1, 25)	LSTM(25, 25, 1)	101.0	31.6	147.5	85.9	——	——	——	——
Hidden Size	(1, 1, 50)	LSTM(50, 50, 1)	84.4	4.5	33.0	8.3	84.4	4.5	33.0	8.3
	(1, 1, 100)	LSTM(100, 100, 1)	318.7	7.0	96.2	13.8	——	——	——	——
	(1, 1, 150)	LSTM(150, 150, 1)	709.4	9.6	198.6	19.3	——	——	——	——
	(1, 1, 200)	LSTM(200, 200, 1)	1256.2	12.1	340.0	24.8	——	——	——	——
Output Size	(1, 1, 25)	LSTM(25, 25, 2)	54.3	6.3	35.7	10.7	54.3	6.3	35.7	10.7
	(1, 1, 25)	LSTM(25, 25, 3)	80.6	8.8	52.9	15.5	80.6	8.8	52.9	15.5
	(1, 1, 25)	LSTM(25, 25, 4)	106.9	11.1	70.1	20.5	106.9	11.1	70.1	20.5
	(1, 1, 25)	LSTM(25, 25, 5)	133.2	13.2	87.3	25.1	133.2	13.2	87.3	25.1
	(1, 1, 25)	LSTM(25, 25, 6)	159.4	15.4	104.5	30.1	——	——	——	——

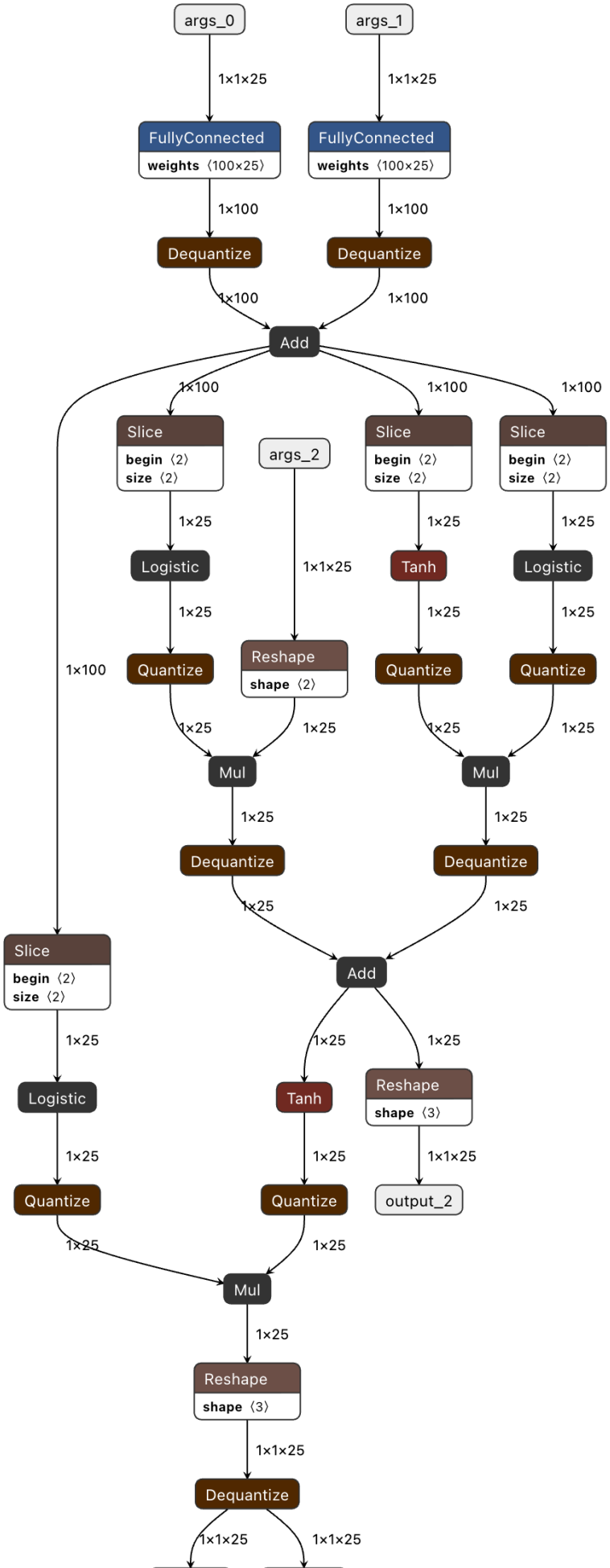

args_0
args_1
1×1×25
1×1×25
FullyConnected
weights ⟨100×25⟩
FullyConnected
weights ⟨100×25⟩
1×100
1×100
Dequantize
Dequantize
1×100
1×100
Add
1×100
1×100
1×100
Slice
begin ⟨2⟩
size ⟨2⟩
args_2
Slice
begin ⟨2⟩
size ⟨2⟩
Slice
begin ⟨2⟩
size ⟨2⟩
1×25
1×25
1×25
Logistic
1×1×25
Tanh
Logistic
1×25
1×25
1×25
1×100
Quantize
Reshape
shape ⟨2⟩
Quantize
Quantize
1×25
1×25
1×25
1×25
Mul
Mul
1×25
1×25
Dequantize
Dequantize
1×25
1×25
Slice
begin ⟨2⟩
size ⟨2⟩
Add
1×25
1×25
1×25
Logistic
Tanh
Reshape
shape ⟨3⟩
1×25
1×25
1×1×25
Quantize
Quantize
output_2
1×25
1×25
Mul
1×25
Reshape
shape ⟨3⟩
1×1×25
Dequantize
1×1×25
1×1×25

Table 4.3 reports our benchmark of the manually implemented LSTM layers using the default LiteRT export and quantization pipeline. Inspecting the exported model layout reveals that the dominant source of int8 latency overhead is the set of intermittent `Quantize`/`Dequantize` stages (Fig. 4.4), which must traverse the entire activation tensors at each transition. The cost of these stages scales with the number of such traversals per inference: in the time-step and layer-count sweeps, where these transitions recur, int8 inference is consistently slower than its float32 counterpart (e.g., 17538 μs vs. 9542 μs at 15 time steps on PSRAM). Notably, this trend reverses for large single-layer hidden sizes, where the matrix computation dominates the quantization overhead and int8 becomes faster than float32 (e.g., 27723 μs vs. 42847 μs for a hidden size of 200 on PSRAM).

A second observation concerns quantization quality. Single-layer, single-time-step configurations incur negligible error (MSE = 0), whereas accuracy degrades once additional time steps or stacked layers are introduced: the mean-squared error rises to 0.0036 at 15 time steps and reaches 0.0050 for the four-layer configuration. This indicates that the accumulation of quantization error across recurrent and stacked computations, rather than the LSTM operation in isolation, is the principal driver of the observed accuracy loss.

The memory footprint (Table 4.4) shows the opposite balance to the convolutional case. Because LSTM weight matrices scale quadratically with the hidden size, the model is parameter-dominated, and int8 quantization produces large reductions in model size: for a hidden size of 200 the model shrinks from 1256.2 KB (float32) to 340.0 KB (int8), a 3.7× saving. The arena, however, is consistently *larger* under int8 than under float32 (e.g. 12.1 KB → 24.8 KB at hidden size 200, and 31.6 KB → 85.9 KB at 20 time steps), reflecting the additional scratch buffers required by the intermittent `Quantize`/`Dequantize` stages discussed above. Arena requirements also grow with the number of time steps (3.2 → 31.6 KB for float32 from one to twenty steps), which, together with the inflated int8 arena, explains why several int8 configurations exceed the available internal SRAM and could only be executed from PSRAM (denoted by dashes in Table 4.3).

4.2. Comparison of different Model Configurations

Table 4.5.: Model A (float32) performance on the ESP32-S3.

Win	Step	Stage A (ticks)	ELU-A (µs)	Stage B (ticks)	ELU-B (µs)	LSTM step (ticks)	Reg step (ticks)	Window (µs)	MSE
50	1	146 269 [146 376]	1 630 [1 941]	3 756 [3 765]	779.1 [933]	5 274 [6 010]	172.3 [186]	298 232 [298 772]	6.72×10^{-14} [1.95×10^{-13}]
	5	146 290 [146 383]	1 627 [1 942]	3 737 [3 746]	776.8 [928]	26 227 [26 838]	449.4 [466]	293 022 [293 484]	6.74×10^{-14} [1.96×10^{-13}]
100	1	317 014 [317 033]	3 715 [4 223]	9 178 [9 186]	1 549 [1 808]	5 256 [6 062]	172.1 [189]	624 679 [625 634]	3.72×10^{-14} [1.80×10^{-13}]
	5	316 976 [316 995]	3 657 [4 163]	9 241 [9 252]	1 551 [1 810]	26 166 [26 905]	457.6 [474]	614 633 [615 428]	3.73×10^{-14} [1.80×10^{-13}]
150	1	487 975 [487 991]	6 035 [6 775]	14 994 [15 003]	2 318 [2 734]	5 248 [6 046]	177.4 [193]	954 941 [956 454]	7.49×10^{-14} [3.02×10^{-13}]
	5	487 983 [487 997]	6 028 [6 768]	14 902 [14 911]	2 320 [2 742]	26 136 [26 877]	449.9 [467]	937 156 [938 391]	7.49×10^{-14} [3.01×10^{-13}]
200	1	658 646 [658 664]	8 114 [9 055]	20 362 [20 371]	3 183 [3 682]	5 253 [6 042]	171.5 [185]	1 282 398 [1 284 181]	4.94×10^{-14} [1.74×10^{-13}]
	5	658 640 [658 657]	8 118 [9 062]	20 361 [20 373]	3 182 [3 681]	26 154 [26 898]	451.7 [468]	1 259 537 [1 261 121]	4.95×10^{-14} [1.75×10^{-13}]
250	1	829 258 [829 274]	10 224 [11 396]	25 657 [25 666]	4 073 [4 707]	5 251 [6 034]	171.6 [184]	1 609 298 [1 611 503]	6.33×10^{-14} [1.67×10^{-13}]
	5	829 251 [829 269]	10 137 [11 312]	25 662 [25 673]	4 072 [4 707]	26 151 [26 897]	451.9 [466]	1 580 642 [1 582 549]	6.34×10^{-14} [1.67×10^{-13}]
300	1	999 946 [999 958]	12 146 [13 549]	30 700 [30 706]	4 928 [5 689]	5 247 [6 051]	173.0 [188]	1 934 628 [1 937 212]	5.19×10^{-14} [1.51×10^{-13}]
	5	999 941 [999 965]	12 145 [13 550]	30 703 [30 708]	4 952 [5 716]	26 105 [26 866]	453.8 [470]	1 901 027 [1 903 361]	5.21×10^{-14} [1.52×10^{-13}]
350	1	1 170 798 [1 170 813]	14 121 [15 741]	35 837 [35 850]	5 793 [6 684]	5 245 [6 047]	173.4 [186]	2 261 084 [2 263 981]	5.60×10^{-14} [1.57×10^{-13}]
	5	1 171 059 [1 171 075]	14 116 [15 743]	35 816 [35 832]	5 795 [6 679]	26 082 [26 837]	457.5 [474]	2 220 702 [2 223 437]	5.59×10^{-14} [1.58×10^{-13}]
400	1	1 341 979 [1 341 993]	16 140 [18 034]	40 889 [40 897]	6 637 [7 647]	5 246 [6 053]	172.8 [187]	2 587 306 [2 590 543]	6.04×10^{-14} [1.69×10^{-13}]
	5	1 342 217 [1 342 234]	16 136 [18 034]	40 891 [40 899]	6 638 [7 649]	26 108 [26 879]	447.4 [461]	2 541 540 [2 544 316]	6.05×10^{-14} [1.69×10^{-13}]
450	1	1 513 510 [1 513 530]	18 198 [20 285]	45 952 [45 961]	7 478 [8 662]	5 244 [6 034]	172.2 [185]	2 914 544 [2 918 399]	4.86×10^{-14} [1.29×10^{-13}]
	5	1 513 770 [1 513 791]	18 195 [20 282]	45 958 [45 968]	7 481 [8 664]	26 102 [26 873]	446.1 [461]	2 862 572 [2 866 126]	4.86×10^{-14} [1.29×10^{-13}]
500	1	1 685 167 [1 685 176]	20 210 [22 505]	51 188 [51 194]	8 314 [9 563]	5 245 [6 042]	173.9 [187]	3 244 354 [3 248 458]	4.49×10^{-14} [1.46×10^{-13}]
	5	1 685 472 [1 685 488]	20 206 [22 499]	51 067 [51 077]	8 316 [9 565]	26 095 [26 848]	446.6 [462]	3 185 937 [3 189 677]	4.50×10^{-14} [1.47×10^{-13}]

Table 4.6.: Model A (int8 A,B, ELU stages) performance on the ESP32-S3.

Win	Step	Stage A (ticks)	ELU-A (µs)	Stage B (ticks)	ELU-B (µs)	LSTM step (ticks)	Reg step (ticks)	Window (µs)	MSE
50	1	8 983 [9 135]	124.8 [130]	978.2 [988]	768.2 [858]	5 236 [5 923]	170.9 [192]	155 388 [155 616]	9.97×10^{-6} [2.02×10^{-4}]
	5	8 957 [9 104]	124.9 [130]	987.6 [997]	768.9 [855]	26 149 [26 733]	445.3 [464]	151 061 [151 275]	9.97×10^{-6} [2.02×10^{-4}]
100	1	19 728 [19 860]	244.8 [250]	1 681 [1 689]	1 495 [1 649]	5 211 [5 967]	172.2 [189]	310 719 [311 079]	8.04×10^{-6} [7.86×10^{-5}]
	5	19 754 [19 859]	245.1 [250]	1 681 [1 688]	1 494 [1 645]	26 099 [26 809]	435.1 [454]	302 606 [302 900]	8.04×10^{-6} [7.86×10^{-5}]
150	1	29 843 [29 879]	365.0 [370]	2 466 [2 473]	2 156 [2 326]	5 209 [6 002]	171.9 [189]	467 877 [468 234]	3.32×10^{-5} [2.47×10^{-4}]
	5	29 869 [29 883]	365.0 [370]	2 495 [2 502]	2 157 [2 325]	26 066 [26 851]	436.5 [459]	453 731 [454 222]	3.32×10^{-5} [2.47×10^{-4}]
200	1	39 912 [39 932]	484.9 [490]	3 702 [3 711]	2 861 [3 061]	5 207 [6 028]	170.2 [191]	623 713 [624 244]	1.71×10^{-4} [3.06×10^{-3}]
	5	39 919 [39 937]	485.0 [490]	3 705 [3 714]	2 861 [3 056]	26 110 [26 893]	443.6 [460]	606 909 [607 476]	1.71×10^{-4} [3.06×10^{-3}]
250	1	49 907 [49 925]	605.0 [610]	5 013 [5 018]	3 673 [4 007]	5 204 [6 030]	170.6 [185]	779 848 [780 596]	2.22×10^{-5} [1.29×10^{-4}]
	5	49 909 [49 925]	605.0 [610]	5 018 [5 024]	3 674 [4 003]	26 098 [26 888]	443.9 [461]	758 860 [759 496]	2.22×10^{-5} [1.29×10^{-4}]
300	1	59 936 [59 950]	725.3 [730]	6 403 [6 410]	4 394 [4 705]	5 208 [6 054]	171.9 [186]	938 146 [938 941]	1.08×10^{-5} [4.64×10^{-5}]
	5	59 940 [59 953]	724.9 [730]	6 425 [6 433]	4 388 [4 696]	26 098 [26 921]	441.4 [460]	911 326 [912 154]	1.08×10^{-5} [4.64×10^{-5}]
350	1	69 968 [69 981]	845.4 [847]	8 186 [8 192]	5 426 [5 931]	5 209 [6 035]	171.1 [184]	1 096 215 [1 097 103]	4.45×10^{-5} [4.52×10^{-4}]
	5	69 645 [69 660]	847.8 [853]	8 153 [8 160]	5 427 [5 936]	26 046 [26 854]	442.8 [460]	1 062 569 [1 063 453]	4.45×10^{-5} [4.52×10^{-4}]
400	1	79 771 [79 786]	967.6 [973]	9 678 [9 682]	6 195 [6 646]	5 218 [6 054]	176.1 [190]	1 257 044 [1 257 998]	1.66×10^{-5} [5.32×10^{-5}]
	5	79 800 [79 809]	988.5 [993]	9 868 [9 871]	6 196 [6 646]	26 036 [26 848]	439.2 [453]	1 215 080 [1 216 192]	1.66×10^{-5} [5.32×10^{-5}]
450	1	89 985 [90 002]	1 126 [1 131]	11 429 [11 434]	6 927 [7 384]	5 213 [6 058]	174.2 [188]	1 414 842 [1 415 962]	2.29×10^{-5} [5.57×10^{-5}]
	5	90 024 [90 038]	1 131 [1 135]	11 423 [11 431]	6 928 [7 381]	26 039 [26 866]	452.5 [474]	1 368 701 [1 369 799]	2.29×10^{-5} [5.57×10^{-5}]
500	1	100 235 [100 249]	1 295 [1 300]	12 741 [12 746]	7 709 [8 213]	5 196 [6 031]	172.1 [185]	1 563 943 [1 565 309]	1.72×10^{-5} [3.83×10^{-5}]
	5	100 262 [100 275]	1 315 [1 319]	12 738 [12 744]	7 708 [8 214]	26 043 [26 872]	451.9 [468]	1 521 132 [1 522 487]	1.72×10^{-5} [3.83×10^{-5}]

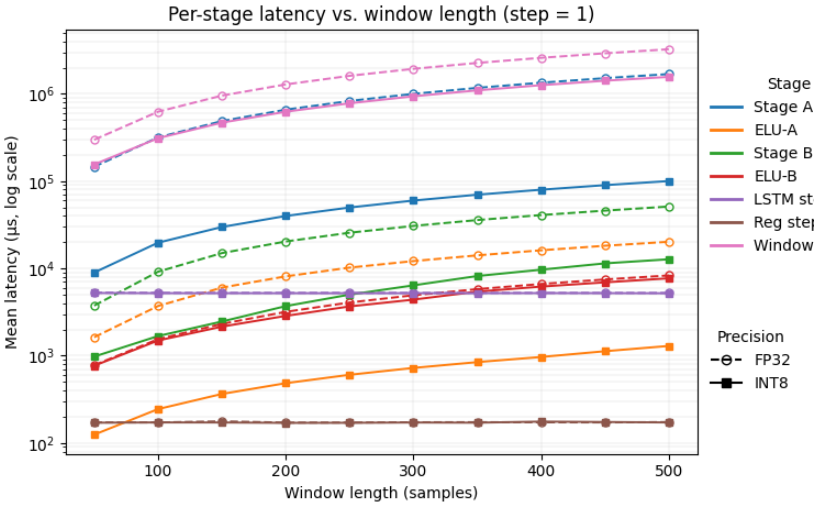


Figure 4.5.: Figure shows latency of model stages over window length.

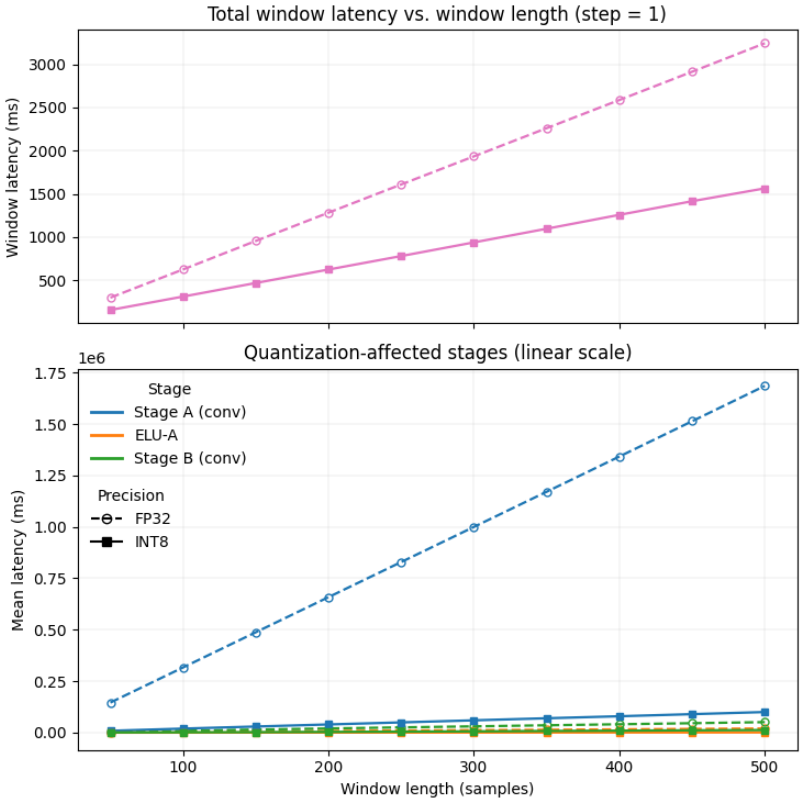


Figure 4.6.: Figure shows latency of model stages over window length.

Table 4.7.: Model B (float32) performance on the ESP32-S3.

Win	Step	Stage A (ticks)	ELU-A (µs)	Stage B (ticks)	ELU-B (µs)	LSTM step (ticks)	Reg step (ticks)	Window (µs)	MSE
50	1	61 619 [61 691]	1 552 [1 852]	3 742 [3 756]	781.4 [976]	5 276 [6 016]	104.0 [120]	211 747 [212 107]	1.76×10^{-14} [7.12×10^{-14}]
	5	61 643 [61 698]	1 549 [1 849]	3 733 [3 744]	782.1 [976]	26 241 [26 855]	265.1 [283]	207 413 [207 782]	1.76×10^{-14} [7.20×10^{-14}]
100	1	131 484 [131 498]	3 458 [3 951]	9 229 [9 239]	1 557 [1 883]	5 250 [6 070]	106.9 [122]	435 189 [435 908]	7.84×10^{-15} [3.16×10^{-14}]
	5	131 463 [131 485]	3 452 [3 951]	9 129 [9 139]	1 559 [1 885]	26 165 [26 902]	265.4 [281]	426 575 [427 208]	7.77×10^{-15} [3.15×10^{-14}]
150	1	201 608 [201 622]	5 739 [6 431]	14 995 [15 003]	2 323 [2 783]	5 258 [6 051]	106.5 [117]	662 879 [663 917]	2.06×10^{-14} [1.03×10^{-13}]
	5	201 572 [201 590]	5 818 [6 509]	14 962 [14 971]	2 325 [2 781]	26 154 [26 905]	265.7 [279]	648 405 [649 171]	2.05×10^{-14} [9.88×10^{-14}]
200	1	271 398 [271 411]	7 808 [8 681]	20 417 [20 429]	3 207 [3 824]	5 245 [6 053]	107.0 [120]	886 303 [887 561]	1.54×10^{-14} [4.29×10^{-14}]
	5	271 393 [271 409]	7 806 [8 679]	20 416 [20 430]	3 209 [3 822]	26 151 [26 900]	264.2 [276]	868 500 [869 613]	1.51×10^{-14} [4.27×10^{-14}]
250	1	341 182 [341 195]	9 759 [10 787]	25 661 [25 671]	4 099 [4 868]	5 243 [6 041]	107.0 [122]	1 109 739 [1 111 317]	3.27×10^{-14} [1.01×10^{-13}]
	5	341 174 [341 192]	9 747 [10 779]	25 668 [25 679]	4 101 [4 860]	26 151 [26 894]	264.1 [280]	1 087 617 [1 088 936]	3.28×10^{-14} [1.01×10^{-13}]
300	1	410 952 [410 965]	11 670 [12 942]	30 727 [30 734]	4 997 [5 873]	5 251 [6 051]	108.3 [120]	1 335 628 [1 337 795]	1.57×10^{-14} [5.00×10^{-14}]
	5	410 947 [410 964]	11 686 [12 961]	30 730 [30 738]	4 998 [5 871]	26 179 [26 948]	270.6 [286]	1 307 814 [1 309 409]	1.56×10^{-14} [4.98×10^{-14}]
350	1	480 885 [480 898]	13 581 [15 041]	35 840 [35 852]	5 808 [6 796]	5 260 [6 046]	108.7 [120]	1 561 859 [1 564 457]	2.87×10^{-14} [7.95×10^{-14}]
	5	480 886 [480 905]	13 575 [15 030]	35 836 [35 844]	5 811 [6 800]	26 149 [26 902]	270.2 [283]	1 524 853 [1 526 735]	2.86×10^{-14} [7.93×10^{-14}]
400	1	551 236 [551 247]	15 517 [17 118]	40 902 [40 907]	6 651 [7 770]	5 267 [6 054]	105.8 [119]	1 786 600 [1 789 213]	2.40×10^{-14} [5.78×10^{-14}]
	5	551 178 [551 198]	15 513 [17 116]	41 011 [41 018]	6 652 [7 769]	26 101 [26 850]	264.8 [278]	1 743 573 [1 745 654]	2.40×10^{-14} [5.67×10^{-14}]
450	1	621 814 [621 826]	17 485 [19 292]	46 083 [46 094]	7 494 [8 737]	5 252 [6 060]	107.0 [117]	2 008 382 [2 010 839]	1.52×10^{-14} [5.84×10^{-14}]
	5	621 779 [621 797]	17 481 [19 290]	46 089 [46 097]	7 496 [8 736]	26 117 [26 865]	270.0 [284]	1 963 818 [1 966 185]	1.50×10^{-14} [5.60×10^{-14}]
500	1	692 580 [692 596]	19 432 [21 383]	51 198 [51 204]	8 331 [9 699]	5 242 [6 039]	109.4 [120]	2 230 667 [2 234 169]	2.83×10^{-14} [1.03×10^{-13}]
	5	692 530 [692 548]	19 429 [21 387]	51 176 [51 190]	8 334 [9 696]	26 109 [26 858]	268.0 [280]	2 181 579 [2 184 160]	2.84×10^{-14} [1.04×10^{-13}]

Table 4.8.: Model B (int8 A,B, ELU stages) performance on the ESP32-S3.

Win	Step	Stage A (ticks)	ELU-A (µs)	Stage B (ticks)	ELU-B (µs)	LSTM step (ticks)	Reg step (ticks)	Window (µs)	MSE
50	1	124 615 [124 692]	126.4 [132]	1 006 [1 017]	730.9 [817]	5 231 [5 889]	99.89 [114]	268 640 [268 892]	1.86×10^{-4} [1.59×10^{-3}]
	5	124 637 [124 683]	126.4 [132]	1 013 [1 022]	730.0 [814]	26 121 [26 683]	253.2 [263]	265 258 [265 459]	1.86×10^{-4} [1.59×10^{-3}]
100	1	283 042 [283 064]	246.4 [252]	1 729 [1 737]	1 392 [1 495]	5 219 [5 985]	99.94 [115]	569 697 [570 057]	1.04×10^{-4} [7.38×10^{-4}]
	5	283 045 [283 058]	246.4 [252]	1 734 [1 740]	1 391 [1 494]	26 066 [26 775]	251.8 [261]	562 877 [563 204]	1.04×10^{-4} [7.38×10^{-4}]
150	1	440 921 [440 937]	366.6 [372]	2 548 [2 555]	2 075 [2 195]	5 215 [6 010]	99.98 [107]	872 259 [872 750]	1.20×10^{-4} [6.33×10^{-4}]
	5	440 916 [440 936]	366.4 [372]	2 552 [2 559]	2 074 [2 193]	26 043 [26 806]	251.9 [260]	860 635 [861 086]	1.20×10^{-4} [6.33×10^{-4}]
200	1	598 739 [598 749]	486.5 [492]	3 750 [3 759]	2 744 [2 907]	5 213 [6 028]	100.0 [110]	1 173 963 [1 174 545]	3.11×10^{-5} [2.33×10^{-4}]
	5	598 732 [598 741]	486.6 [492]	3 755 [3 759]	2 743 [2 902]	26 032 [26 802]	252.0 [261]	1 158 411 [1 158 971]	3.11×10^{-5} [2.33×10^{-4}]
250	1	756 552 [756 563]	606.8 [612]	5 136 [5 142]	3 521 [3 705]	5 211 [6 031]	100.0 [108]	1 475 980 [1 476 607]	1.86×10^{-4} [1.10×10^{-3}]
	5	756 546 [756 554]	606.7 [612]	5 139 [5 143]	3 492 [3 672]	26 025 [26 806]	252.0 [261]	1 456 487 [1 457 157]	1.86×10^{-4} [1.10×10^{-3}]
300	1	914 356 [914 363]	726.8 [732]	6 537 [6 542]	4 235 [4 362]	5 210 [6 045]	100.0 [109]	1 778 522 [1 779 473]	1.44×10^{-4} [6.30×10^{-4}]
	5	914 349 [914 360]	727.4 [732]	6 540 [6 549]	4 199 [4 326]	26 018 [26 819]	251.9 [260]	1 755 115 [1 755 848]	1.44×10^{-4} [6.30×10^{-4}]
350	1	1 072 167 [1 072 175]	847.7 [853]	8 246 [8 255]	5 133 [5 331]	5 209 [6 045]	100.1 [109]	2 081 939 [2 082 782]	5.36×10^{-5} [4.46×10^{-4}]
	5	1 072 160 [1 072 168]	853.7 [859]	8 181 [8 185]	5 200 [5 398]	26 014 [26 816]	252.0 [264]	2 054 671 [2 055 538]	5.36×10^{-5} [4.46×10^{-4}]
400	1	1 230 155 [1 230 160]	976.2 [981]	9 943 [9 948]	6 012 [6 294]	5 208 [6 036]	100.1 [107]	2 384 843 [2 386 013]	9.01×10^{-5} [5.20×10^{-4}]
	5	1 230 150 [1 230 162]	972.3 [974]	9 866 [9 870]	6 012 [6 286]	26 012 [26 818]	251.9 [260]	2 353 563 [2 354 659]	9.01×10^{-5} [5.20×10^{-4}]
450	1	1 388 209 [1 388 219]	1 120 [1 125]	11 476 [11 482]	6 693 [6 961]	5 208 [6 029]	100.1 [107]	2 687 388 [2 688 321]	4.90×10^{-5} [3.86×10^{-4}]
	5	1 388 208 [1 388 222]	1 131 [1 135]	11 477 [11 481]	6 692 [6 960]	26 008 [26 819]	252.1 [264]	2 652 416 [2 653 659]	4.90×10^{-5} [3.86×10^{-4}]
500	1	1 546 315 [1 546 324]	1 319 [1 323]	12 788 [12 796]	7 498 [7 841]	5 208 [6 028]	100.1 [107]	2 990 424 [2 991 550]	1.71×10^{-4} [8.66×10^{-4}]
	5	1 546 311 [1 546 322]	1 358 [1 363]	12 790 [12 795]	7 498 [7 840]	26 008 [26 820]	251.9 [263]	2 951 507 [2 952 770]	1.71×10^{-4} [8.66×10^{-4}]

5. Discussion

5.1. Model Performance

The results confirm that `int8` quantization yields substantial latency reductions for the convolutional inference stages on the ESP32-S3. Comparing the fully floating-point configuration (Table 4.5) with the `int8` configuration (Table 4.6), the Stage A inference cost falls from approximately 146,269 to 8,983 profiler ticks at a window length of 50, a speed-up of roughly 16×, while Stage B improves by a factor of about 3.8×. This disparity reflects the substantial gap between the integer arithmetic units and the floating-point unit on the ESP32-S3, and is further attributable to the ESP-NN backend, whose assembly-optimized kernels on this platform target `int8` operations exclusively, leaving floating-point paths to the generic, unoptimized implementations. As expected, quantization introduces a measurable degradation in numerical fidelity: the mean squared error rises from the order of 10^{-14} in the floating-point model—a value consistent with floating-point rounding noise rather than algorithmic error—to the range of approximately 8.0×10^{-6} to 1.7×10^{-4} under `int8` quantization. We argue that this degradation remains tolerable given the magnitude of the accompanying latency improvements.

The ELU activations represent a second area of concern. The TensorFlow Lite Micro runtime and the ESP-NN backend implement only a limited set of activation functions (such as ReLU, Hard-Swish, Logistic, and Softmax) and provide no native implementation for ELU. Consequently, the LiteRT converter decomposes the `torch.nn.ELU` module into a sequence of primitive operations. In the quantized graph, the exporter further isolates this region as a floating-point island bracketed by quantization and dequantization stages that traverse the entire activation tensor, adding considerable overhead. To address this, we removed the ELU section from the model graph and reimplemented it manually using an `int8` look-up table (LUT). For the first activation block (ELU-A), this substitution reduces the latency from approximately 1,630 to 125 µs at a window length of 50 and from 20,211 to 1,295 µs at a window length of 500, corresponding to a speed-up of roughly 13 to 16×. We note that the second activation block (ELU-B) is retained in floating point (dequantized from `int8` without a LUT) and therefore exhibits no comparable improvement, with its latency remaining essentially unchanged between the two configurations (for example, 779 versus 768 µs at a window length of 50).

The LSTM step size constitutes an important hyperparameter governing the trade-off between recurrent loop overhead and memory footprint. Because the multi-layer LSTM is unrolled into its constituent operations, retaining all unrolled timesteps is infeasible: the activation arena grows rapidly with the number of timesteps processed per invocation. This increase affects not only the static memory footprint but potentially the latency as well, since larger arenas may require allocation in the slower external PSRAM or flash, incurring additional memory-access latency. Increasing the step size batches more timesteps into shared buffers,

thereby amortizing the recurrent loop overhead. In our measurements, raising the LSTM step size from 1 to 5 approximately triples the arena reserved for the LSTM stage, from 8.86 to 24.77 kB (a factor of 2.79). The step size must therefore be tuned to the deployment scenario and the data dimensionality at hand in order to maximize performance within a tolerable memory budget.

Window length plays a decisive role in overall pipeline latency and is arguably the single most influential contributor to the end-to-end timing. Our results demonstrate a near-linear relationship between window length and the time required to produce an inference over a complete window: in the floating-point configuration, the mean per-window latency increases from approximately 298,232 µs at a window length of 50 to 3,244,354 µs at a window length of 500, corresponding to an essentially constant per-sample cost of roughly 6,000 µs.

Finally, our results indicate that quantization of the LSTM and regression stages is counterproductive, offering minimal returns relative to the implementation difficulty. Unlike feedforward architectures such as dense and convolutional layers, the LSTM is recurrent: its output at a given timestep feeds the computation at the next, which causes quantization error to compound across timesteps, so that even small errors at the initial timestep accumulate into substantial deviations at the final output. The same effect applies to the regression head, which is invoked within the same recurrent loop. Cell-state quantization is particularly problematic. In contrast to the hidden state, the cell state exhibits a wide dynamic range—spanning approximately -30 to 30 in our experiments—which causes standard `int8` quantization to be coarse: numerically significant variations are collapsed into narrow ranges, producing repeated saturated values and severely degraded error metrics. The conventional remedy is a dedicated `int16` quantization scheme for the cell-state path, which is supported neither by the LiteRT converter nor by the ESP-NN kernels. This would require hand-written operations for the elementwise multiplications and additions, together with more memory-intensive LUTs for activations such as tanh (64 kB each, compared with 256 bytes for `int8` data). The literature on LSTM quantization remains sparse, with the few available studies recommending quantization-aware training methods, which lie beyond the scope of this work.

6. Conclusion

6.1. Conclusion

6.2. Limitations

Several limitations of this work merit discussion.

First, the scope of this thesis was confined to the implementation of the provided model architecture, which was adopted without modification. This is a relevant consideration, as it has been argued that purely convolutional (TCN) architectures can be more reliable than recurrent (RNN) architectures for capturing temporal dependencies in sEMG data [14]; a systematic comparison between the two paradigms on the target hardware therefore remains outside the scope of the present work.

Second, we deliberately adopted a cross-platform approach, seeking to avoid tying the pipeline to any single hardware target. Although all experiments were conducted on ESP32 chips, the pipeline is readily generalisable, as it relies on the industry-standard LiteRT framework. While LiteRT and TensorFlow Lite Micro map operations to native kernel backends, the final performance could nonetheless benefit substantially from platform-specific optimisations that this work did not pursue.

Third, although several LSTM quantisation schemes were investigated, none yielded reliable results. This outcome is attributable to a number of factors that warrant further investigation before faster and more dependable quantisation of recurrent layers can be achieved.

Finally, because the model was supplied pre-trained, quantisation-aware training (QAT) was not explored in depth. Several studies nonetheless report considerable benefits of QAT over simpler post-training quantisation (PTQ) schemes, suggesting a promising direction for improvement. Additionally, alternative quantisation schemes such as hybrid and int16 quantisation were not examined, owing to their incompatibility with the Espressif platforms targeted in this work.

6.3. Future Work

The limitations identified above suggest several promising directions for future research.

First, the deployment pipeline could be evaluated across a broader range of model architectures. In particular, a systematic comparison between the recurrent (LSTM-based) model studied here and a purely convolutional (TCN) counterpart, conducted directly on the target hardware, would clarify the trade-offs between accuracy, memory footprint, and inference latency for each paradigm under realistic embedded constraints.

Second, while the cross-platform design of the pipeline favours portability, future work could quantify the performance gains attainable through platform-specific optimisation. Deploying the pipeline on hardware with dedicated neural-network acceleration, or substituting hand-optimised kernels for the default backend, would establish how much additional efficiency can be recovered without sacrificing the framework's generality.

Third, the difficulties encountered in quantising the recurrent layers motivate a dedicated investigation into reliable quantisation of LSTM and other RNN-based components. Identifying the sources of instability, and developing or adopting quantisation strategies tailored to recurrent computation, would directly address one of the central implementation challenges of this work.

Finally, the quantisation methodology itself could be extended. Incorporating quantisation-aware training (QAT), provided the training pipeline is made available, may yield accuracy improvements over the post-training quantisation employed here. Likewise, evaluating alternative schemes such as hybrid and int16 quantisation on hardware platforms that support them would help characterise the full design space of accuracy-versus-efficiency trade-offs for embedded sEMG regression.

A. Additionally

List of Figures

List of Tables

Bibliography

[1] R. Martinek, M. Ladrova, M. Sidikova, R. Jaros, K. Behbehani, R. Kahankova and A. Kawala-Sterniuk, "Advanced Bioelectrical Signal Processing Methods: Past, Present and Future Approach—Part I: Cardiac Signals," *Sensors*, vol. 21, no. 15, p. 5186, Jul. 2021.

[2] M. Zheng, M. Crouch and M. Eggleston, "Surface Electromyography as a Natural Human-Machine Interface: A Review," *IEEE Sensors Journal*, vol. 22, pp. 1–1, May 2022.

[3] I. V. Valcheva, V. Gandhi and E. Chinellato, "Surface EMG driven gesture recognition using machine learning for robotic applications," *Discover Robotics*, vol. 2, no. 1, p. 4, Feb. 2026.

[4] M. Raez, M. Hussain and F. Mohd-Yasin, "Techniques of EMG signal analysis: Detection, processing, classification and applications," *Biological Procedures Online*, vol. 8, pp. 11–35, Mar. 2006.

[5] U. Côté Allard, C. L. Fall, A. Drouin, A. Campeau-Lecours, C. Gosselin, K. Glette, F. Laviolette and B. Gosselin, "Deep Learning for Electromyographic Hand Gesture Signal Classification Using Transfer Learning," *IEEE transactions on neural systems and rehabilitation engineering: a publication of the IEEE Engineering in Medicine and Biology Society*, vol. PP, Jan. 2019.

[6] C. J. De Luca, L. Donald Gilmore, M. Kuznetsov and S. H. Roy, "Filtering the surface EMG signal: Movement artifact and baseline noise contamination," *Journal of Biomechanics*, vol. 43, no. 8, pp. 1573–1579, May 2010.

[7] B. Hudgins, P. Parker and R. Scott, "A new strategy for multifunction myoelectric control," *IEEE Transactions on Biomedical Engineering*, vol. 40, no. 1, pp. 82–94, Jan. 1993.

[8] L. H. Smith, L. J. Hargrove, B. A. Lock and T. A. Kuiken, "Determining the Optimal Window Length for Pattern Recognition-Based Myoelectric Control: Balancing the Competing Effects of Classification Error and Controller Delay," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 19, no. 2, pp. 186–192, Apr. 2011.

[9] A. Phinyomark, P. Phukpattaranont and C. Limsakul, "Feature Reduction and Selection for EMG Signal Classification," *Expert Systems with Applications*, vol. 39, pp. 7420–7431, Jun. 2012.

[10] U. Côté-Allard, E. Campbell, A. Phinyomark, F. Laviolette, B. Gosselin and E. Scheme, "Interpreting Deep Learning Features for Myoelectric Control: A Comparison with Handcrafted Features," *Frontiers in Bioengineering and Biotechnology*, vol. 8, p. 158, Mar. 2020.

[11] S. Ni, M. A. A. Al-qaness, A. Hawbani, D. Al-Alimi, M. Abd Elaziz and A. A. Ewees, "A survey on hand gesture recognition based on surface electromyography: Fundamentals, methods, applications, challenges and future trends," *Applied Soft Computing*, vol. 166, p. 112235, Nov. 2024.

[12] P. T., V. K. Elumalai and B. E., "Hand gesture classification framework leveraging the entropy features from sEMG signals and VMD augmented multi-class SVM," *Expert Systems with Applications*, vol. 238, p. 121972, Mar. 2024.

[13] N. Mohapatra, J. Sahoo and G. K. Sahoo, *Hand Gesture Classification Using Surface Electromyography Signals Based on Fusion of Time Domain Features*, Apr. 2024.

[14] M. Zanghieri, S. Benatti, A. Burrello, V. Kartsch, F. Conti and L. Benini, "Robust Real-Time Embedded EMG Recognition Framework Using Temporal Convolutional Networks on a Multicore IoT Processor," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 14, no. 2, pp. 244–256, Apr. 2020.

[15] W. Zhong, X. Jiang, K. Szymaniak, M. Jabbari, C. Ma and K. Nazarpour, "Deep Feature Learning From Electromyographic Signals for Gesture Recognition Systems," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 34, pp. 20–35, 2026.

[16] H. Yelchuri and R. R, "A review of TinyML," 2022.

[17] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan and S. Han, "MCUNet: Tiny Deep Learning on IoT Devices," 2020.

[18] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang and P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," Mar. 2021.

[19] A. Faraone and R. Delgado, "Convolutional-Recurrent Neural Networks on Low-Power Wearable Platforms for Cardiac Arrhythmia Detection," p. 157, Aug. 2020.

[20] E. Kim, J. Kim, J. Park, H. Ko and Y. Kyung, "TinyML-Based Classification in an ECG Monitoring Embedded System," *Computers, Materials & Continua*, vol. 75, pp. 1751–1764, Jan. 2023.

[21] M. Zanghieri, S. Benatti, A. Burrello, V. J. Kartsch Morinigo, R. Meattini, G. Palli, C. Melchiorri and L. Benini, "sEMG-based Regression of Hand Kinematics with Temporal Convolutional Networks on a Low-Power Edge Microcontroller," in *2021 IEEE International Conference on Omni-Layer Intelligent Systems (COINS)*. Barcelona, Spain: IEEE, Aug. 2021, pp. 1–6.

[22] M. Atzori, A. Gijsberts, I. Kuzborskij, S. Elsig, A.-G. M. Hager, O. Deriaz, C. Castellini, H. Muller and B. Caputo, "Characterization of a benchmark database for myoelectric movement classification," *IEEE transactions on neural systems and rehabilitation engineering: a publication of the IEEE Engineering in Medicine and Biology Society*, vol. 23, no. 1, pp. 73–83, Jan. 2015.

[23] Espressif Systems, "ESP32-S3 Technical Reference."

[24] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," Jun. 2018.

[25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," Dec. 2017.

[26] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen and T. Blankevoort, "A White Paper on Neural Network Quantization," Jun. 2021.

Declaration

Herewith, I declare that I have developed and written the enclosed thesis entirely by myself and that I have not used sources or means except those declared.

This thesis has not been submitted to any other authority to achieve an academic grading and has not been published elsewhere.

Stuttgart, TBD Date of sign. Andrew Fady Fahmy